

编译原理

3. 语法分析-自底向上

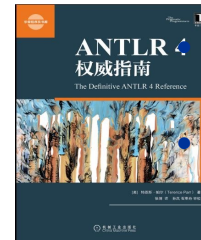
rainoftime.github.io
浙江大学
计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
- 3. Parsing**
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

回顾: Top-Down Parsing

- **LL(1): left-to-right scan** + **leftmost derivation**
 - 从左向右读程序，最左推导，采用1个前看符号
- **自顶向下分析的优点**
 - 直观、符合文法结构
 - **手动构造** or **自动生成**
- **LL(1)分析的局限性**
 - 能分析的文法类型受限



ANTLR3: LL(*) [[PLDI11](#)]

ANTLR4: Adaptive LL(*) [[OOPSLA14](#)]



[Terence Parr](#)



[Sam Harwell](#)



[Kathleen Fisher](#)

Bottom-Up Parsing

- **基于LR(k)文法**(*Knuth*, 1963)
 - 表达力: Every LL(k) grammar is also LR(k)
 - 被Parser自动生成器广泛采用(Yacc, Bison, etc)
- **LR(k) parsing**
 - “**L**”: left-to-right scanning 自左向右扫描
 - “**R**”: rightmost derivation in reverse 最右推导的逆
 - “*k*”: 向前看的token个数(当 *k* 被省略时, 假设为1)

LR(0), SLR(1), LR(1), LALR(1), ...

本讲内容

1

Shift-Reduce

2

LR(0) 分析

3

SLR 分析

1. 移进-规约(Shift-Reduce) (Overview of LR)

The Idea of Bottom-Up Parsing

- **自底向上分析：从串 w 归约为文法开始符号 S**
 - 规约：与某产生式右部相匹配的特定子串，被替换为该产生式头部的非终结符
- **考虑文法 $E \rightarrow E + (E) \mid \text{int}$ 和串 $\text{int} + (\text{int}) + (\text{int})$**
- **两个核心问题**
 1. 何时归约 (归约哪些符号串) ?
 2. 归约到哪个非终结符号 ?

LR分析的一般模式: 移进-规约(Shift-Reduce)

- **Idea:** Split string into two substrings $\alpha \mid \beta$
 - **Right substring** (a string of terminals) is as yet unexamined by parser
 - **Left substring** has terminals and non-terminals
- The dividing point is marked by a $\mid$
 - The $\mid$ is not part of the string
- Initially, all input is unexamined: $\mid x_1 x_2 \dots x_n$

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ Initially, all input is unexamined

int + (int) + (int)

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int + (int) + (int)
↑

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$

int + (int) + (int)
↑

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

int + (int) + (int)
↑

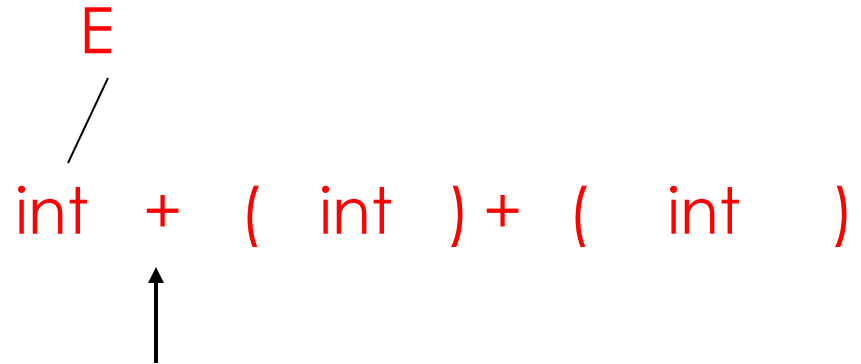
例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$



例: 移进-规约 Shift-Reduce

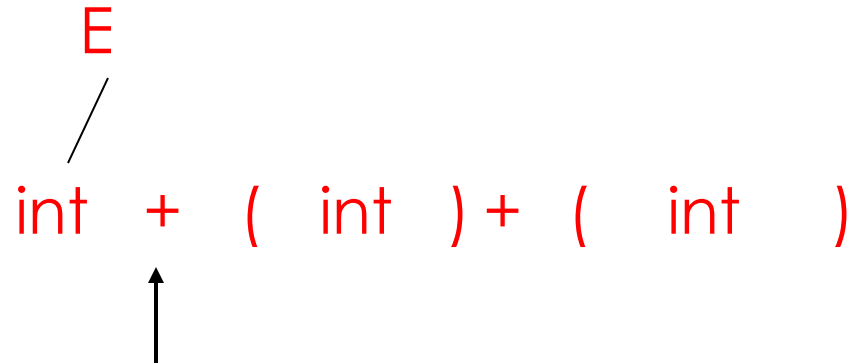
$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$ shift 3 times

先不要管 “为什么选择shift 3次”



例: 移进-规约 Shift-Reduce

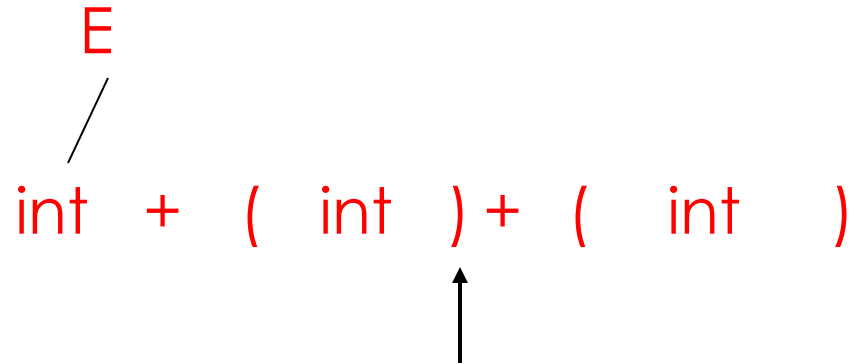
$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$ shift 3 times

E + (int |) + (int)\$



例: 移进-规约 Shift-Reduce

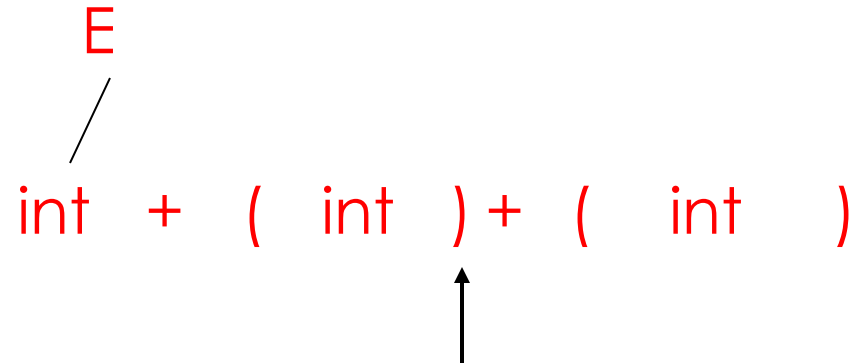
$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$ shift 3 times

E + (int |) + (int)\$ reduce $E \rightarrow \text{int}$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

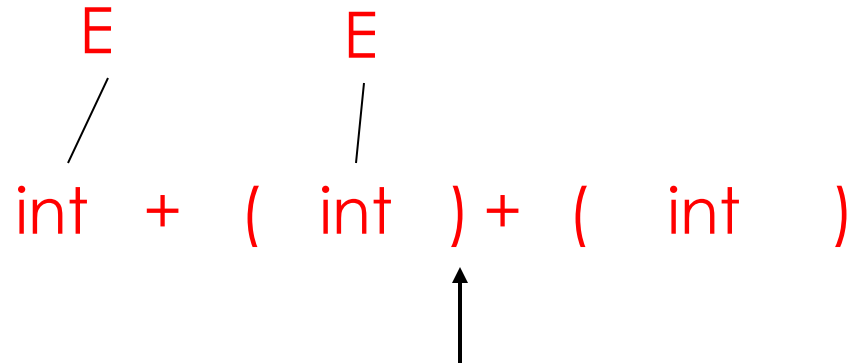
| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$ shift 3 times

E + (int |) + (int)\$ reduce $E \rightarrow \text{int}$

E + (E |) + (int)\$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift

int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$

E | + (int) + (int)\$ shift 3 times

E + (int |) + (int)\$ reduce $E \rightarrow \text{int}$

E + (E |) + (int)\$ shift

$$\begin{array}{ccccccc} & E & & E & & & \\ & / & & / & & & \\ \text{int} & + & (& \text{int} &) & + & (& \text{int} &) \end{array}$$

↑

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$

Diagram illustrating the shift-reduce process for the expression $\text{int} + (\text{int}) + (\text{int})$. The expression is shown with the non-terminal E above the first int and the second int , indicating reductions. An upward arrow points to the closing parenthesis of the second sub-expression, indicating a shift operation.

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

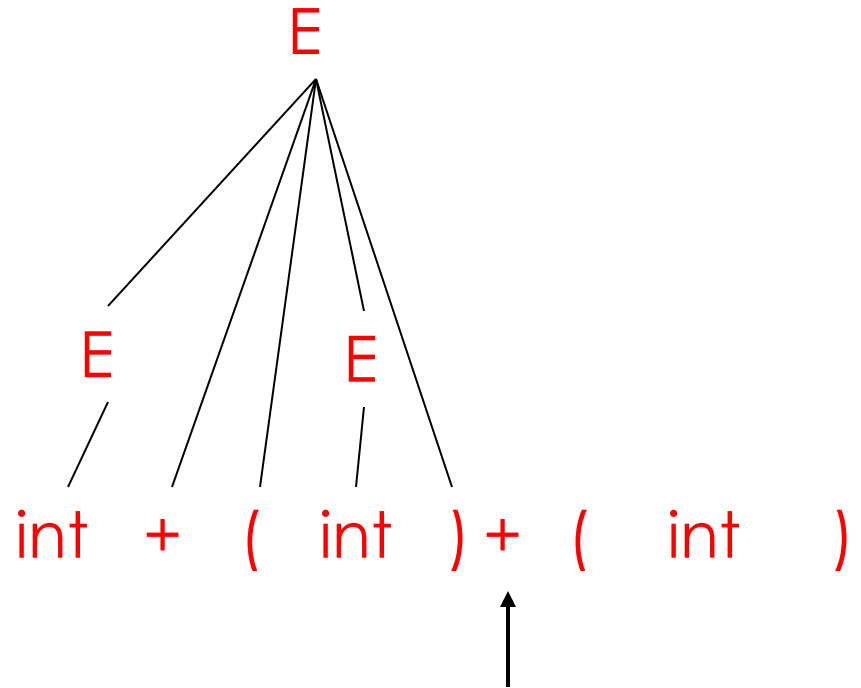
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ reduce $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ reduce $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$

Diagram illustrating the expression $\text{int} + (\text{int}) + (\text{int})$ with non-terminals E above the first and second int tokens, and an upward arrow pointing to the closing parenthesis of the second sub-expression.

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

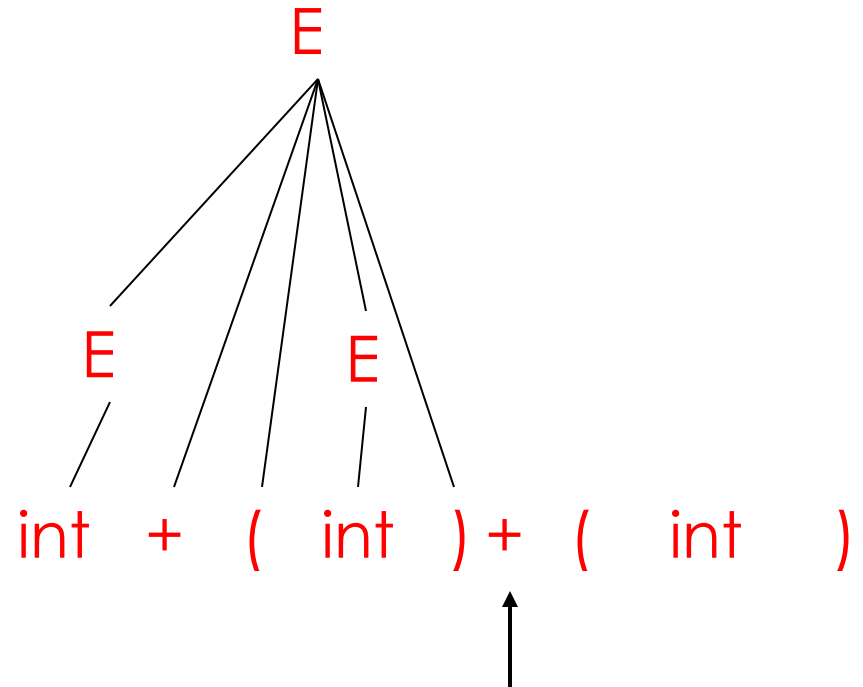
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

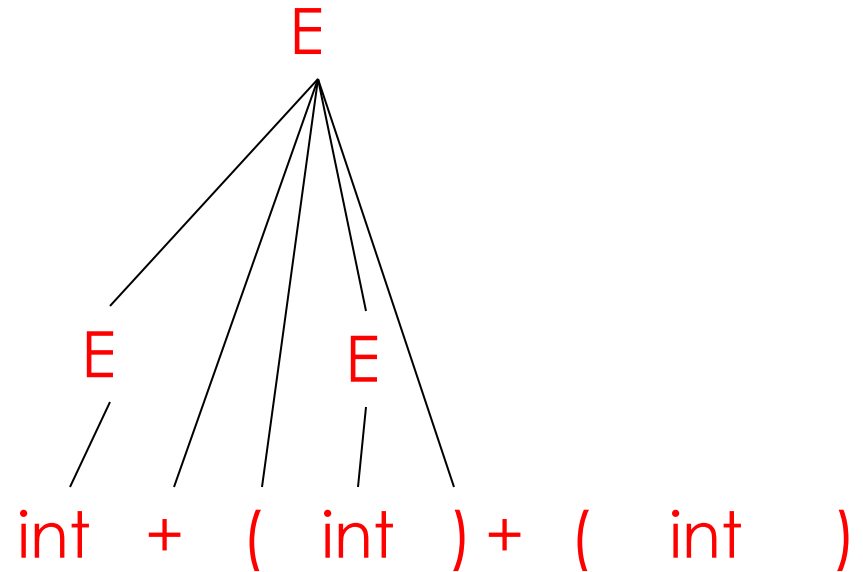
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

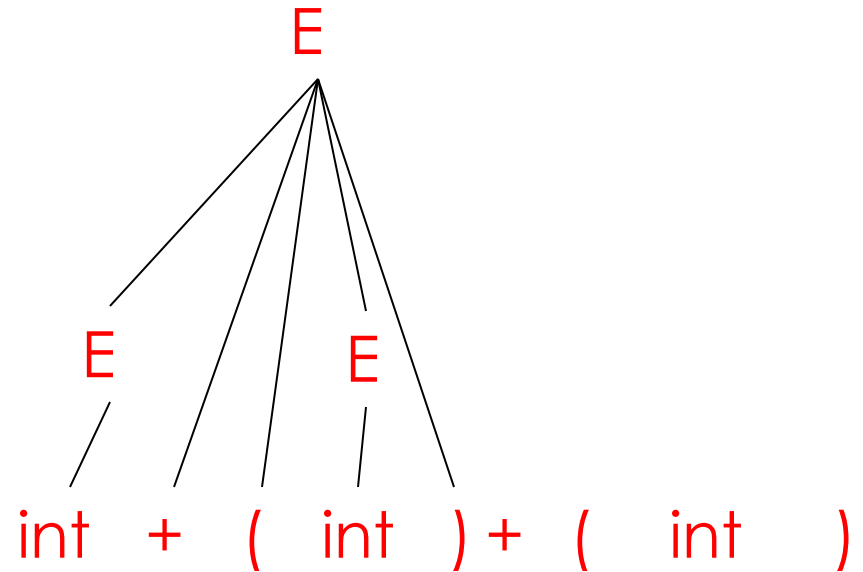
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times
E + (int |)\$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

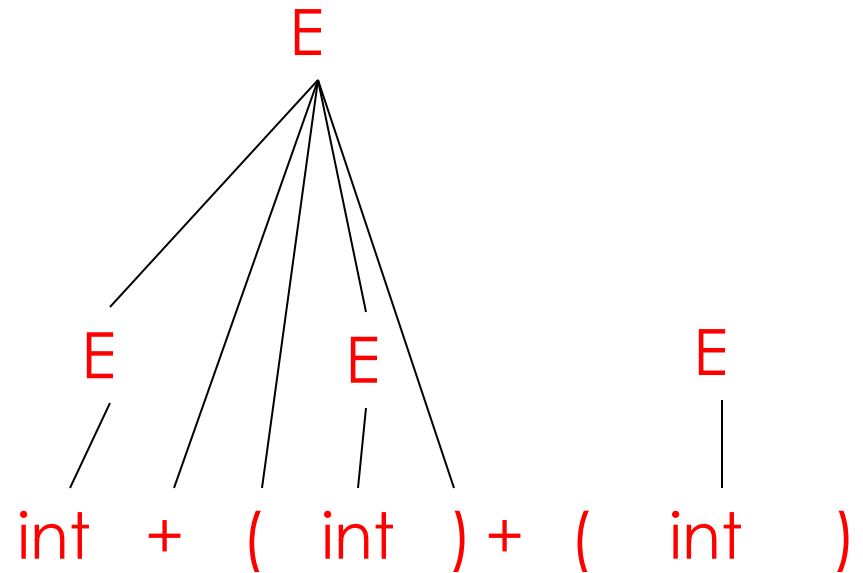
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times
E + (int |)\$ red. $E \rightarrow \text{int}$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

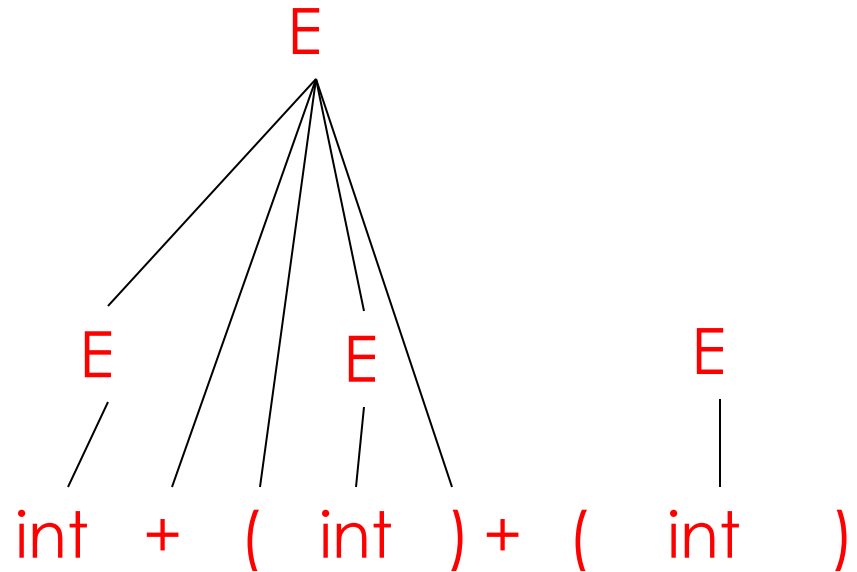
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times
E + (int |)\$ red. $E \rightarrow \text{int}$
E + (E |)\$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

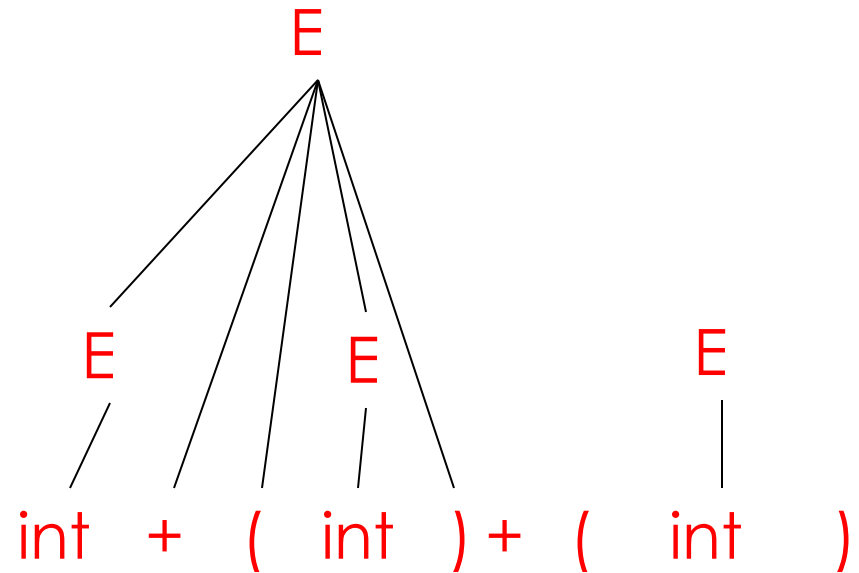
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times
E + (int |)\$ red. $E \rightarrow \text{int}$
E + (E |)\$ shift



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

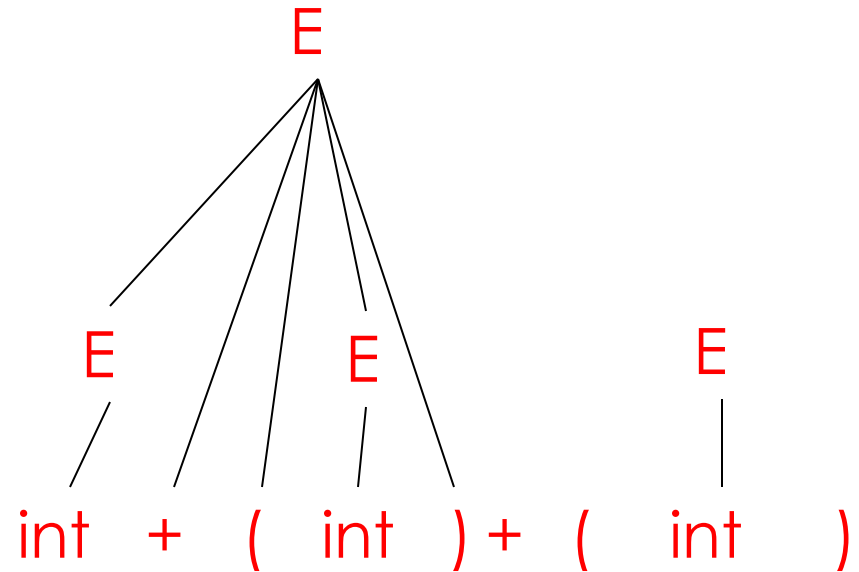
| int + (int) + (int)\$ shift
int | + (int) + (int)\$ red. $E \rightarrow \text{int}$
E | + (int) + (int)\$ shift 3 times
E + (int |) + (int)\$ red. $E \rightarrow \text{int}$
E + (E |) + (int)\$ shift
E + (E) | + (int)\$ red. $E \rightarrow E + (E)$
E | + (int)\$ shift 3 times
E + (int |)\$ red. $E \rightarrow \text{int}$
E + (E |)\$ shift
E + (E) | \$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

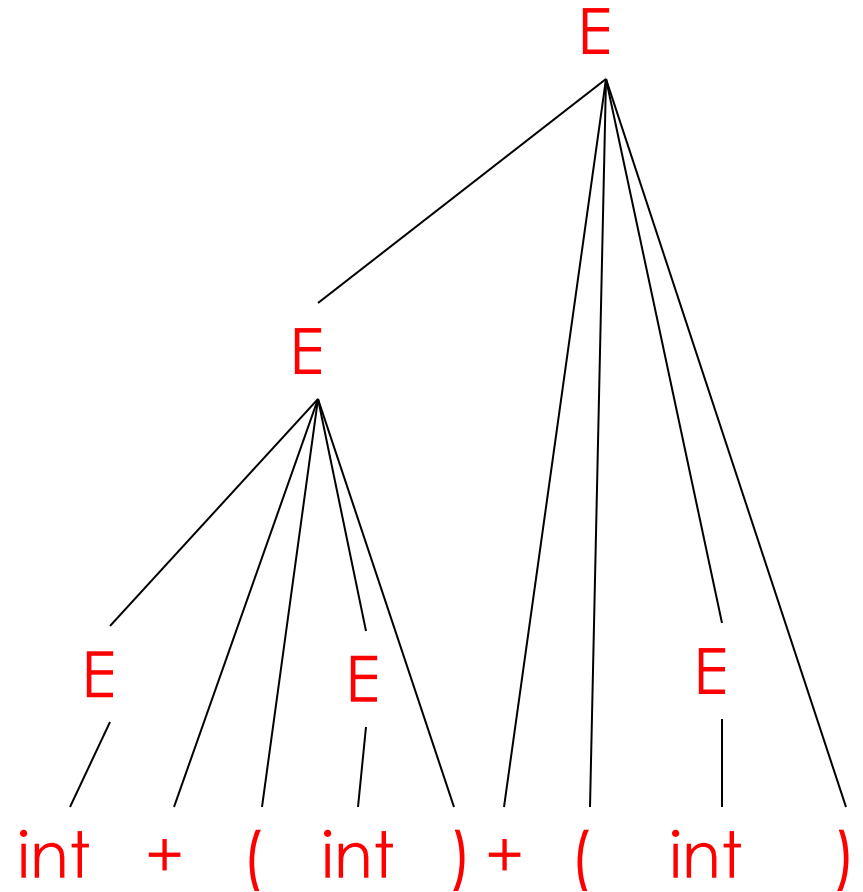
int + (int) + (int)\$	shift
int + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (int) + (int)\$	shift 3 times
E + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (E) + (int)\$	shift
E + (E) + (int)\$	red. $E \rightarrow E + (E)$
E + (int)\$	shift 3 times
E + (int)\$	red. $E \rightarrow \text{int}$
E + (E)\$	shift
E + (E) \$	red. $E \rightarrow E + (E)$



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

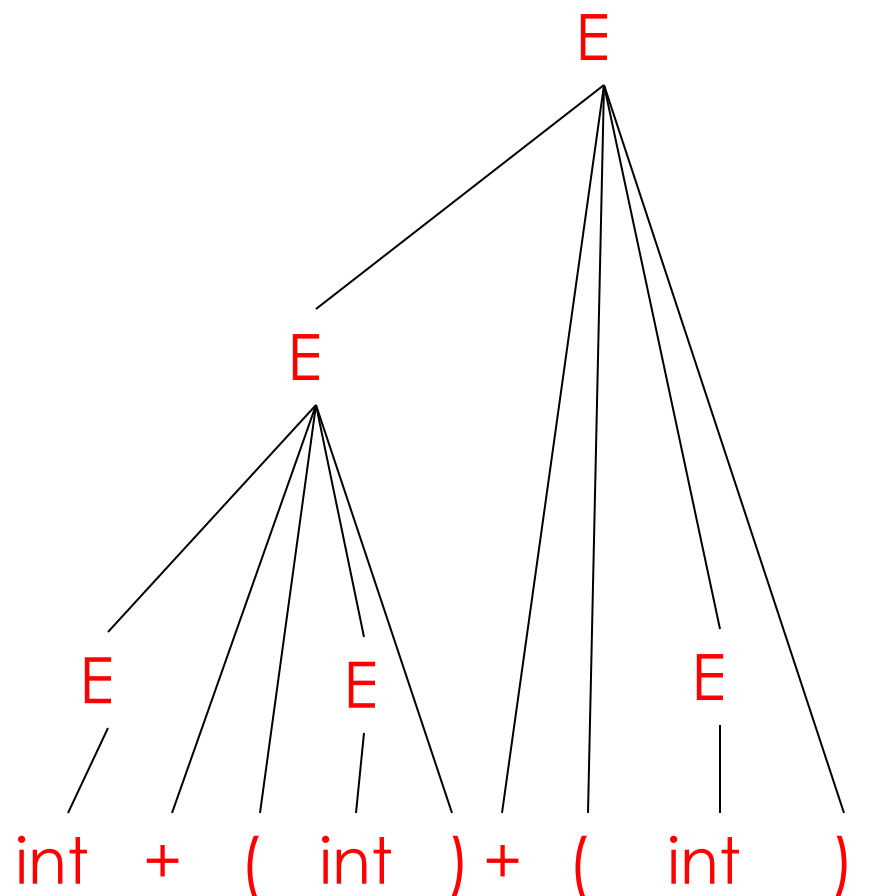
int + (int) + (int)\$	shift
int + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (int) + (int)\$	shift 3 times
E + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (E) + (int)\$	shift
E + (E) + (int)\$	red. $E \rightarrow E + (E)$
E + (int)\$	shift 3 times
E + (int)\$	red. $E \rightarrow \text{int}$
E + (E)\$	shift
E + (E) \$	red. $E \rightarrow E + (E)$
E \$	



例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

int + (int) + (int)\$	shift
int + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (int) + (int)\$	shift 3 times
E + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (E) + (int)\$	shift
E + (E) + (int)\$	red. $E \rightarrow E + (E)$
E + (int)\$	shift 3 times
E + (int)\$	red. $E \rightarrow \text{int}$
E + (E)\$	shift
E + (E) \$	red. $E \rightarrow E + (E)$
E \$	accept

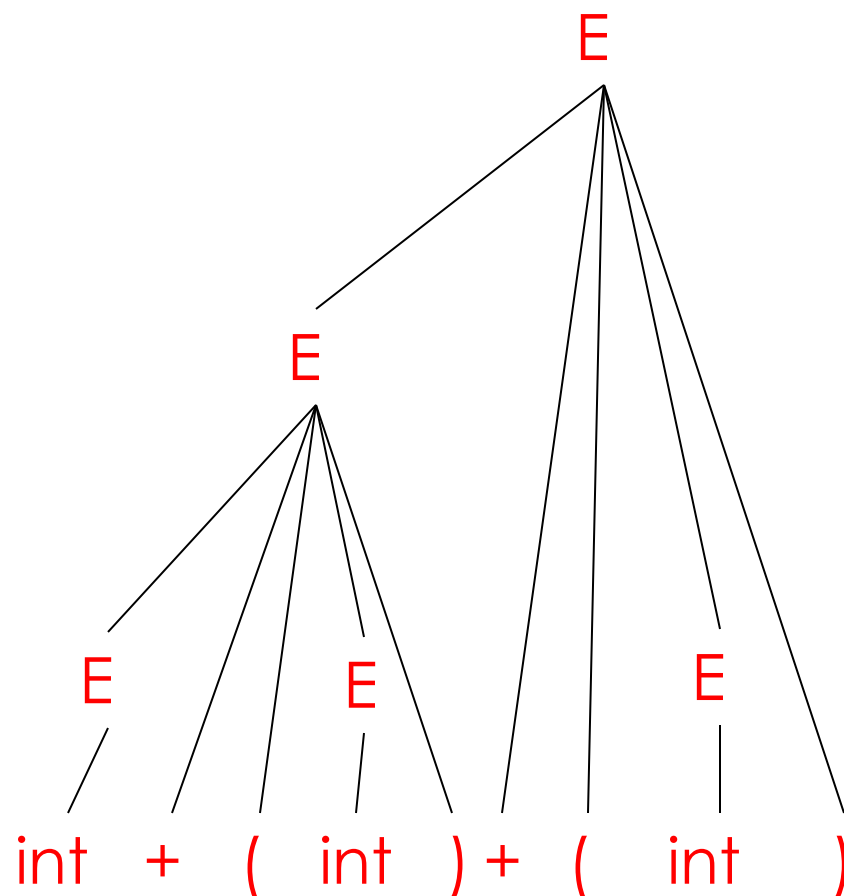


规约到了文法的Start Symbol, i.e., E

例: 移进-规约 Shift-Reduce

$$E \rightarrow E + (E) \mid \text{int}$$

int + (int) + (int)\$	shift
int + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (int) + (int)\$	shift 3 times
E + (int) + (int)\$	red. $E \rightarrow \text{int}$
E + (E) + (int)\$	shift
E + (E) + (int)\$	red. $E \rightarrow E + (E)$
E + (int)\$	shift 3 times
E + (int)\$	red. $E \rightarrow \text{int}$
E + (E)\$	shift
E + (E) \$	red. $E \rightarrow E + (E)$
E \$	accept

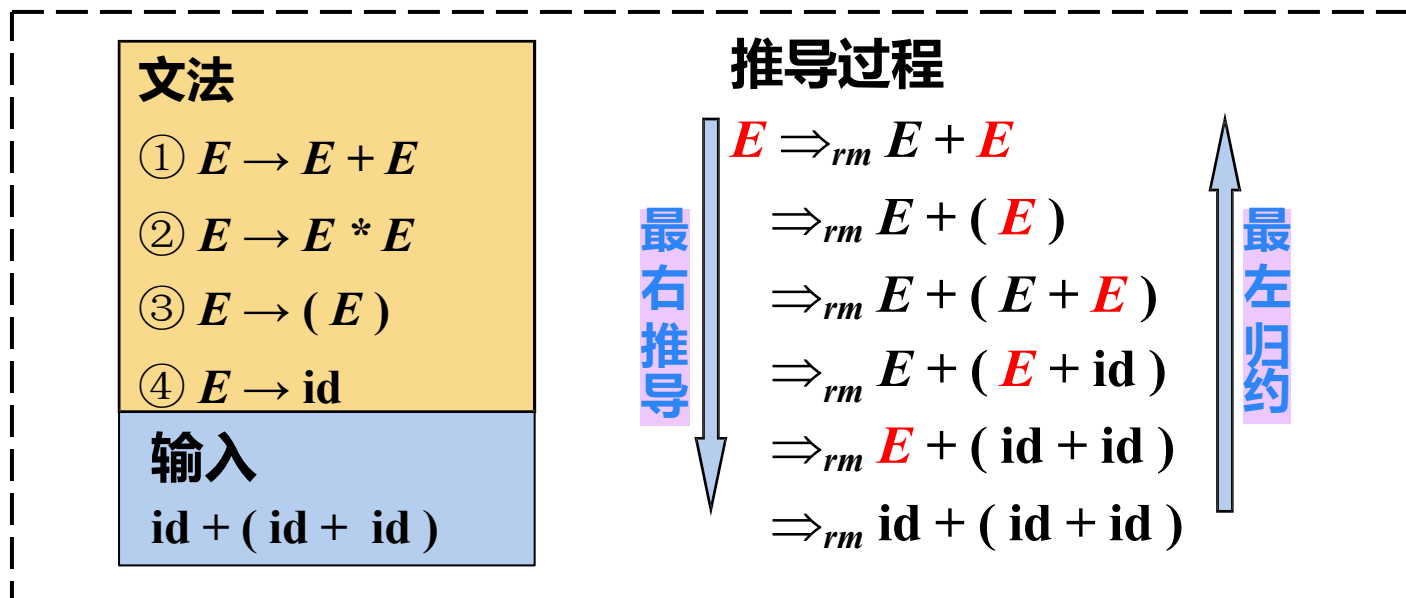


$E \Rightarrow E + (E) \Rightarrow E + (\text{int}) \Rightarrow E + (E) + (\text{int}) \Rightarrow E + (\text{int}) + (\text{int}) \Rightarrow \text{int} + (\text{int}) + (\text{int})$

最右推导的逆过程! (Rightmost derivation in reverse)

LR分析的一般模式: 移进-归约 (Shift-Reduce)

- “完整版”LR分析: 最右推导的逆过程
 - 限制了规约方式(类似LL中的最左推导)
- **最右句型**: 最右推导过程中出现的句型
 - 每一步都是最右句型 $\alpha \mid \beta$



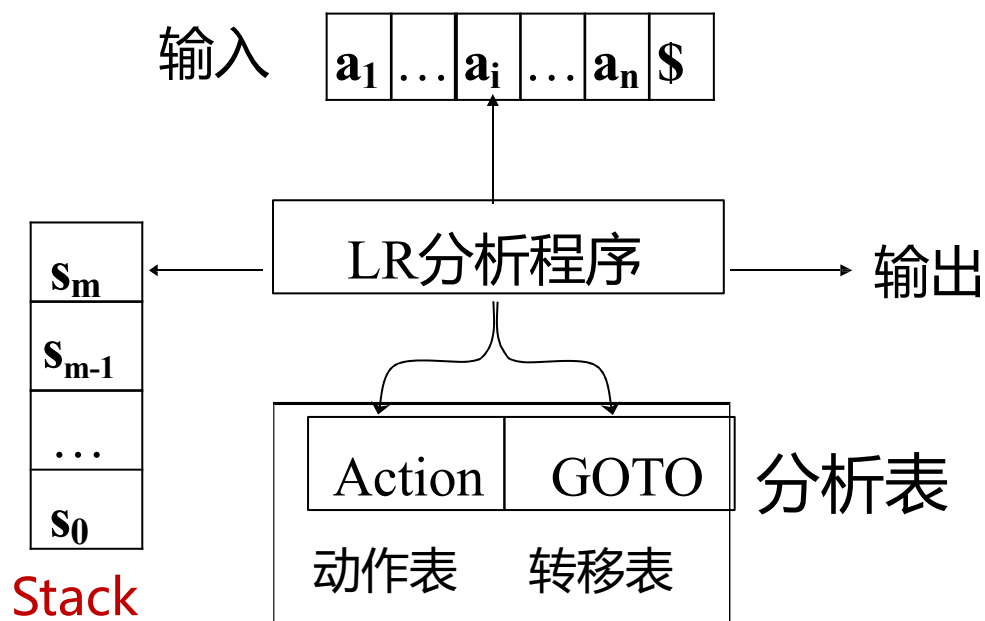
LR分析的实现: 基于栈的移进-归约

- **Two components**

- **Symbol Stack**: holds the left-substring α (terminals + nonterminals)

And the “**state stack**” (will talk about this later)

- **Input stream**: holds the remaining input β (terminals)



- LR系列分析使用相同的表驱动分析程序
- 分析表随文法的不同而各异(如SLR, LR(1))

LR分析的实现: 基于栈的移进-归约

- **Four actions**

- **Shift**: push next input (terminal) on to top of symbol stack

- **Reduce**:

- Top of symbol stack matches RHS of rule (e.g., $X \rightarrow \mathbf{AB}$)

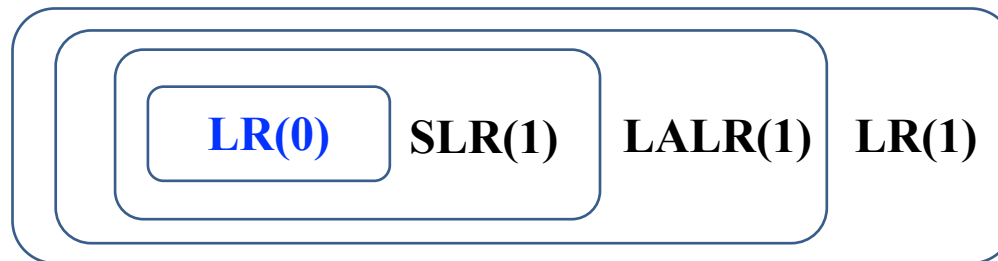
- 1. pop the RHS from the top of stack (e.g., **pop** $\mathbf{BA}$)

- 2. push the LHS onto the stack (e.g., **push** $\mathbf{X}$)

- **Error**

- **Accept**: shift \$ and can reduce what remains on stack to the start symbol!

核心问题: When to shift and when to reduce?



2. LR(0) 分析

- **LR(0) Parsing NFA**
- **LR(0) Parsing DFA**
- **LR(0) Parsing 分析表**
- **LR(0) Parsing 过程**

LR(0) 语法分析思路

- 自底向上分析: 不断地凑出产生式的 RHS
- 假设下一次将会用到的产生式为 $X \rightarrow \alpha \beta$, 使用它进行规约前, **栈顶可能包含3种情况**
 1. 还没有开始匹配这个产生式:
 2. 已经匹配了 α : ... α ...
 3. 已经匹配了 $\alpha \beta$: ... $\alpha \beta$

凑出这个产生式RHS 的进度不同



对于这些情况, 希望对其做出不一样的操作

LR(0) 语法分析思路

- **关键问题:** 如何知道栈顶的内容可以归约了？
 - 维护一个**状态**，记录当前识别的进度！
- **项/Item:** 一个产生式加上在其中某处的一个点
 - 例: 产生式 $A \rightarrow \bullet XYZ$ 有4个Item
 $A \rightarrow \bullet XYZ$, $A \rightarrow X \bullet YZ$, $A \rightarrow XY \bullet Z$, $A \rightarrow XYZ \bullet$

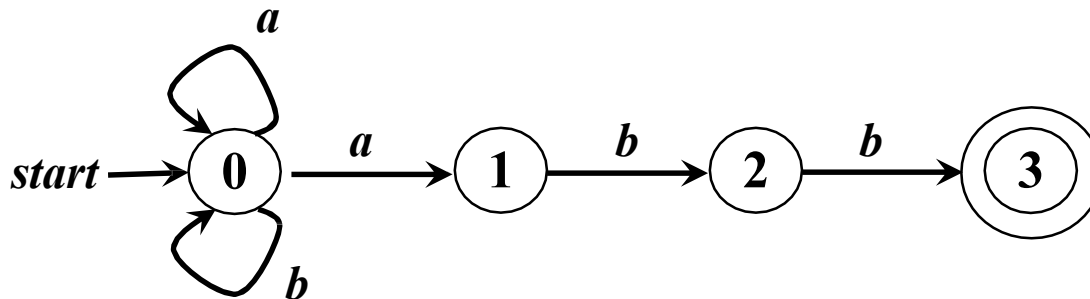
- $A \rightarrow \alpha \bullet \beta$: 已扫描/归约到了 α ，并**期望在接下来的**输入中经过扫描/归约得到 β ，然后把 $\alpha\beta$ 归约到 A
- $A \rightarrow \alpha\beta \bullet$: 已扫描/归约得到了 $\alpha\beta$ ，**可以**把 $\alpha\beta$ 归约为 A

LR(0) Item类似有穷自动机的**状态**!

LR(0) 语法分析思路

- **项/Item:** 一个产生式加上在其中某处的一个点
 - 类似自动机(Automata)里面的状态
- **状态跳转:**
 - 一个项读入一个符号后, 可变为另一个项
 - 如: $A \rightarrow \cdot xyz$ 读入 x 变为 $A \rightarrow x \cdot yz$, 类似状态间的**跳转**

状态 + 跳转 $\rightarrow$ 自动机



自动机案例

LR(0) 语法分析思路

- **项/Item:** 一个产生式加上在其中某处的一个点
 - 类似自动机(Automata)里面的状态
- **状态跳转:**
 - 一个项读入一个符号后, 可变为另一个项
 - 如 $A \rightarrow \cdot xyz$ 读入 x 变为 $A \rightarrow x \cdot yz$, 类似状态间的**跳转**
状态 + 跳转 $\rightarrow$ 自动机
- **项(状态)的数量是有限的**
 - 文法产生式是有限的
 - 每个产生式右部的长度也是有限的

相应的有穷自动机, 被称为**LR(0)自动机**

LR(0) 语法分析思路

- 自底向上分析: 不断地凑出产生式的 RHS
- **关键问题**: 如何知道栈顶的内容可以归约了?
 - 维护一个**状态**, 记录当前识别的进度

➤ 例: $S \rightarrow bBB$

➤ $S \rightarrow \cdot bBB$ ← **移进状态**

➤ $S \rightarrow b \cdot BB$ } **待约状态**

期待B ➤ $S \rightarrow bB \cdot B$ }

➤ $S \rightarrow bBB \cdot$ ← **归约状态**

LR分析器基于这样一些**状态**来构造**自动机**进行RHS的识别

LR(0) Parsing NFA(非确定性有穷自动机)

- 起始&终结状态

- 文法G中增加**新开始符号** S' 并加入产生式 $S' \rightarrow S\$$
- 按 $S' \rightarrow S\$$ 进行归约: 将输入符号串归约成为开始符号 (加入 S' 方便表示起始和终结状态)

$S' \rightarrow S \$$

$S \rightarrow (S)$

$S \rightarrow a$

$S' \rightarrow \bullet S \$$

起始

$S' \rightarrow S \bullet \$$

终止

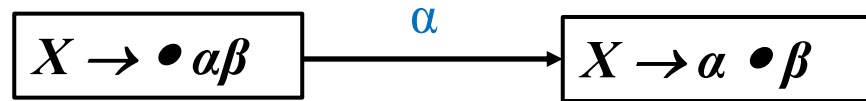
注: 此处的NFA不是指直接用来识别LR(0)语言的自动机 (NFA只能识别正则语言, 然而LR(0)是上下文无关语言)。该NFA是用来“记录当前识别进度”的 (帮助判断栈顶内容是否可归约了)

LR(0) Parsing NFA(非确定性有穷自动机)

- 状态迁移关系

- LR(0) Item之间会有转换关系，如

- 1. $X \rightarrow \bullet \alpha \beta$ 接收 α 后变成 $X \rightarrow \alpha \bullet \beta$

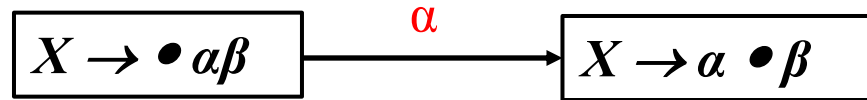


LR(0) Parsing NFA(非确定性有穷自动机)

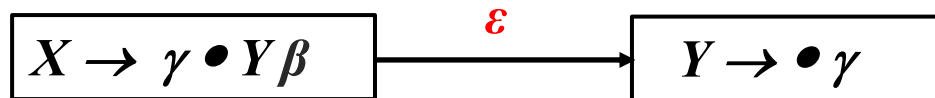
- 状态迁移关系

- LR(0) Item之间会有转换关系，如

1. $X \rightarrow \bullet \alpha \beta$ 接收 α 后变成 $X \rightarrow \alpha \bullet \beta$



2. 如果存在产生式 $X \rightarrow \alpha Y \beta$ 以及 $Y \rightarrow \gamma$,
那么 $X \rightarrow \alpha \bullet Y \beta$ 可以转换到 $Y \rightarrow \bullet \gamma$



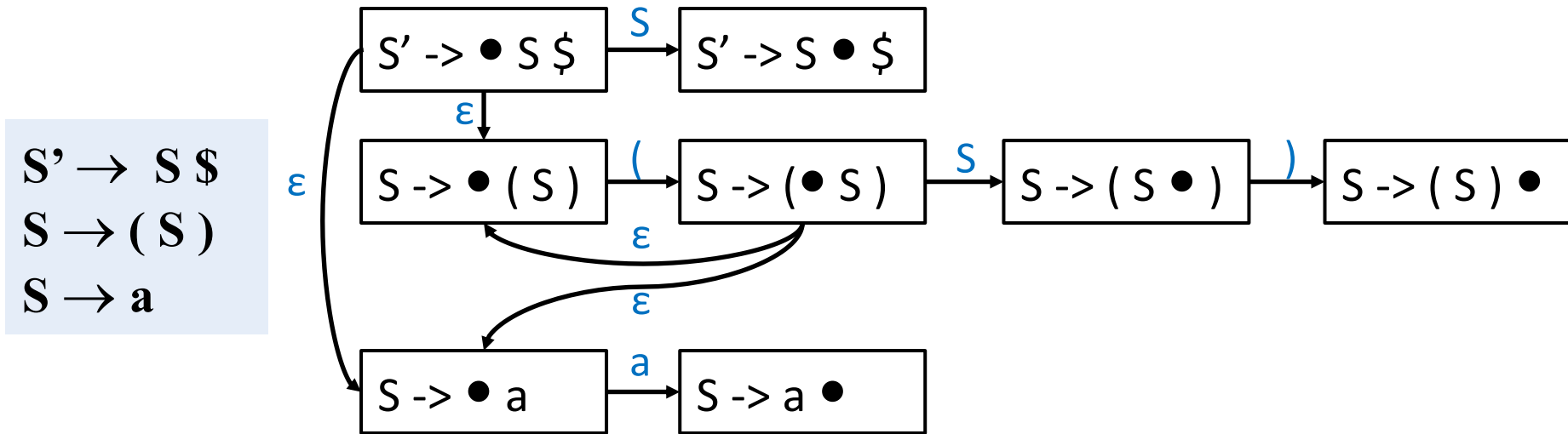
希望看到由 $Y\beta$ 推导出的串, 那要先看到由 Y 推导出的串, 因此加上 Y 的各个产生式对应的项

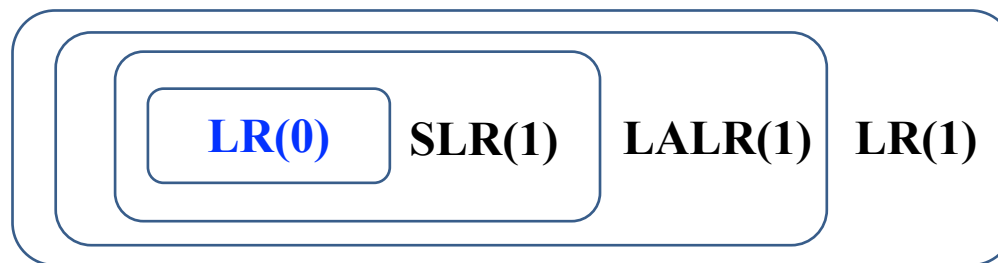
LR(0) Parsing NFA

- 起始&终结状态
- 状态迁移关系

1. $X \rightarrow \bullet \alpha \beta$ 接收 α 后变成 $X \rightarrow \alpha \bullet \beta$
2. 如果存在产生式 $X \rightarrow \alpha Y \beta$, $Y \rightarrow \gamma$, 那么 $X \rightarrow \alpha \bullet Y \beta$ 可以转换到 $Y \rightarrow \bullet \gamma$

利用以上规则构造自动机



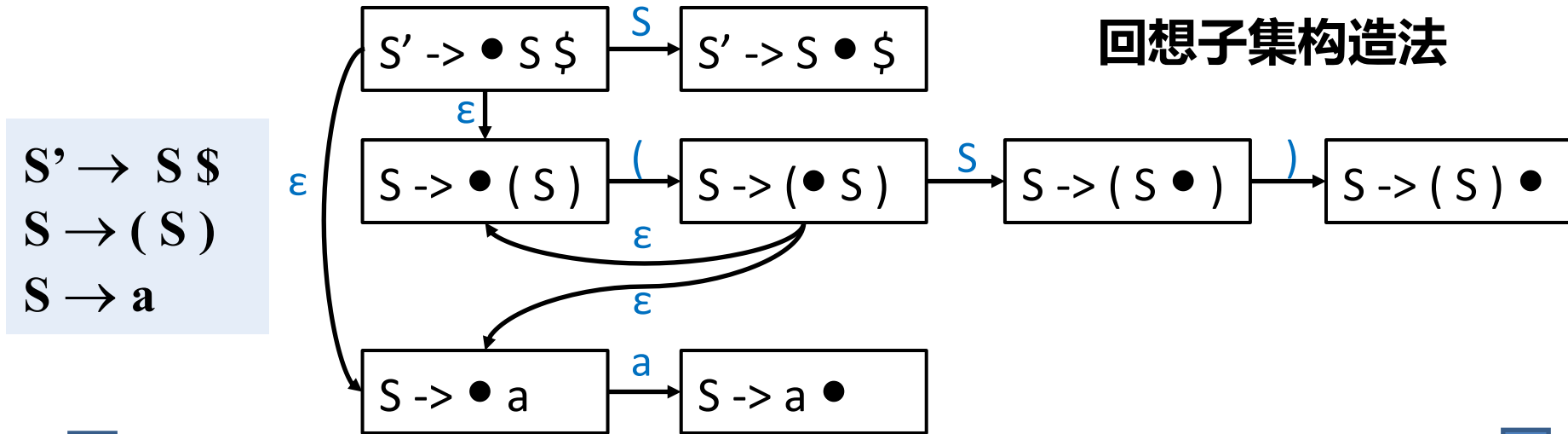


2. LR(0) 分析

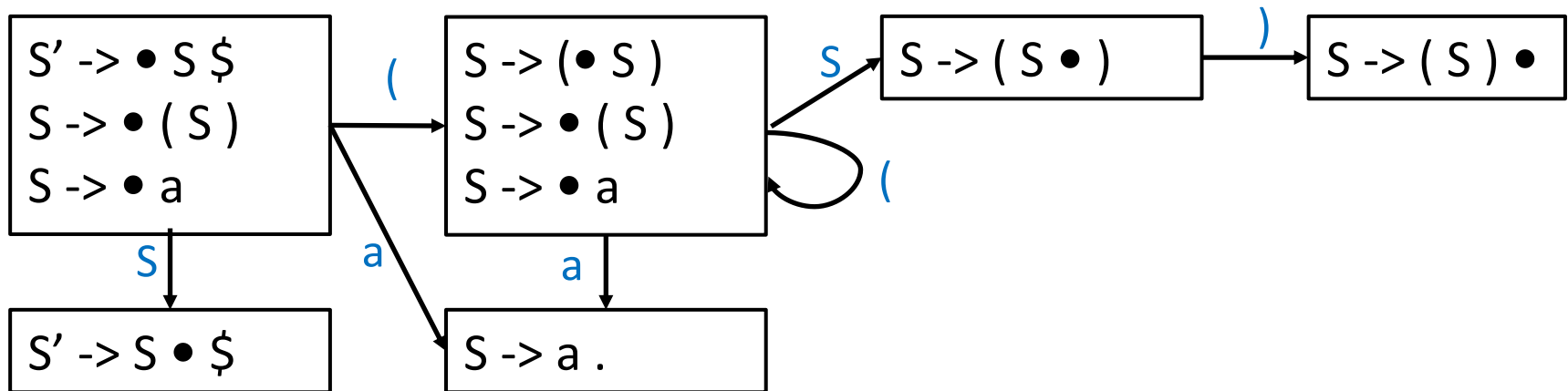
- LR(0) Parsing NFA
- LR(0) Parsing DFA
- LR(0) Parsing 分析表
- LR(0) Parsing 过程

LR(0) Parsing NFA->DFA!

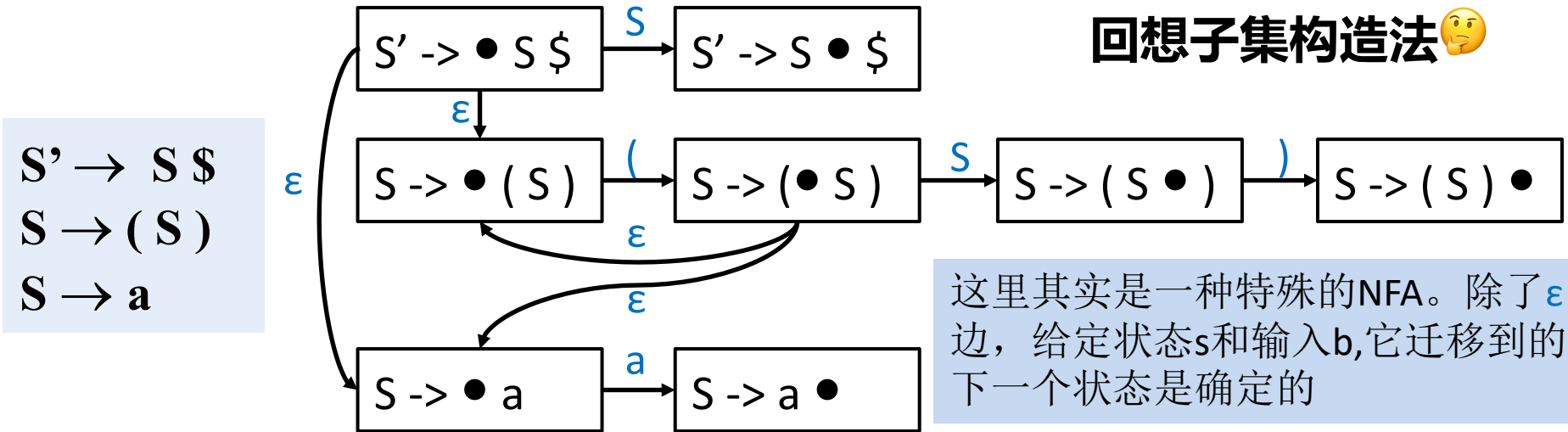
回想子集构造法



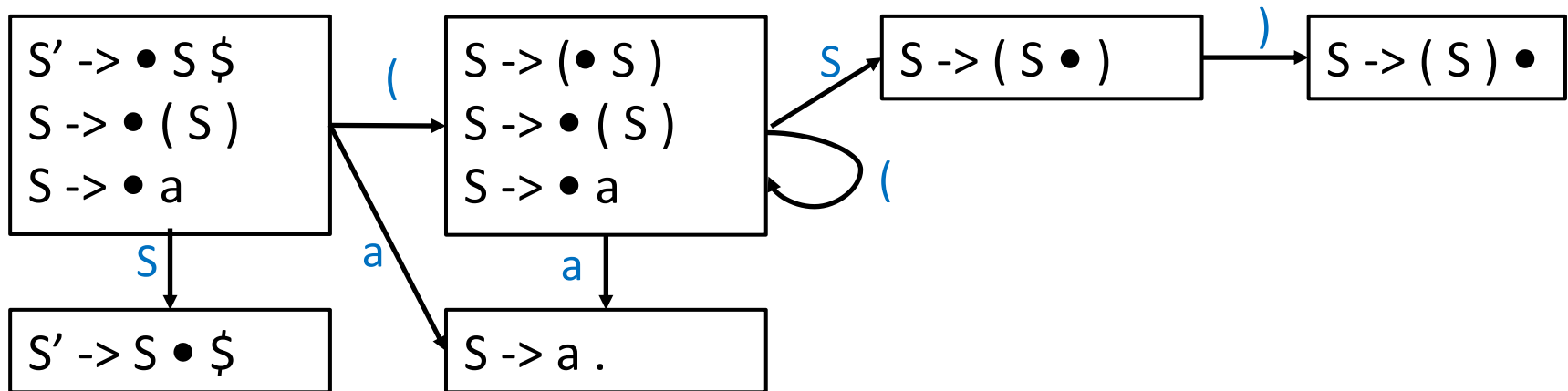
The state of the DFA: **a set of LR(0) items (“项集”)**



LR(0) Parsing NFA->DFA!



The state of the DFA: **a set of LR(0) items (“项集”)**



LR(0)分析通常可以直接构造该DFA

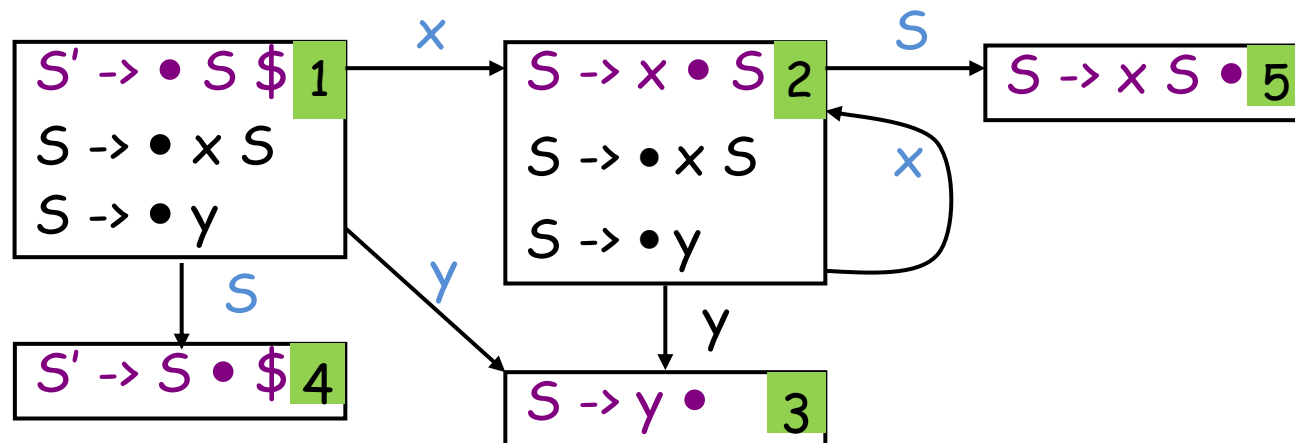
LR(0) Parsing DFA构造

- 如何跳过NFA直接构造以下DFA？

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



LR分析家族一般都是直接构造DFA

LR(0) Parsing DFA构造: Closure(项集闭包)

I: a set of items(项集)

X: a symbol (terminal or non-terminal)

Closure(I) =
 repeat
 for any item $A \rightarrow \alpha \bullet X \beta$ in I
 for any production $X \rightarrow \gamma$
 $I \leftarrow I \cup \{X \rightarrow \bullet \gamma\}$
 until I does not change.
 return I

0: $S' \rightarrow S \$$
1: $S \rightarrow x S$
2: $S \rightarrow y$

$S' \rightarrow \bullet S \$$ 1
 $S \rightarrow \bullet x S$
 $S \rightarrow \bullet y$

左边的项集为

Closure($\{S' \rightarrow \bullet S \$\}$)
(DFA的起始状态)

类似NFA转DFA过程中的 ϵ -Closure

LR(0) Parsing DFA构造: Goto(状态转换函数)

I: a set of items(项集)

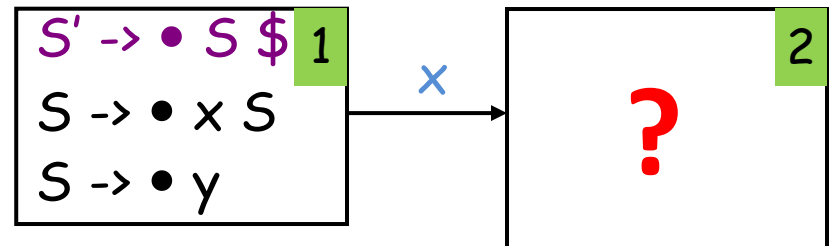
X: a symbol (terminal or non-terminal)

Goto(I, X) =
set J to the empty set
for any item $A \rightarrow \alpha \bullet X \beta$ in I
 add $A \rightarrow \alpha X \bullet \beta$ to J
return **Closure**(J)

类似NFA转DFA中计算新的DFA状态

0: $S' \rightarrow S \$$
1: $S \rightarrow x S$
2: $S \rightarrow y$

项集2 = Goto(项集1, x)



I 是一个项集, X 是一个文法符号, 则Goto(I, X)定义为 I 中所有形如 $A \rightarrow \alpha \cdot X \beta$ 的项所对应的项 $A \rightarrow \alpha X \cdot \beta$ 的集合的闭包

LR(0) Parsing DFA构造: Goto(状态转换函数)

I: a set of items(项集)

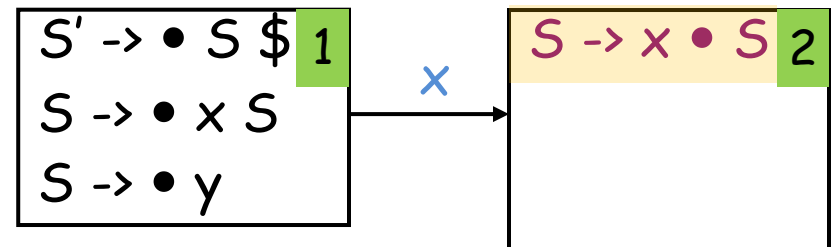
X: a symbol (terminal or non-terminal)

Goto(I, X) =
set J to the empty set
for any item $A \rightarrow \alpha \bullet X \beta$ in I
add $A \rightarrow \alpha X \bullet \beta$ to J
return **Closure**(J)

类似NFA转DFA中计算新的DFA状态

0: $S' \rightarrow S \$$
1: $S \rightarrow x S$
2: $S \rightarrow y$

项集2 = Goto(项集1, x)



I 是一个项集, X 是一个文法符号, 则Goto(I, X)定义为 I 中所有形如 $A \rightarrow \alpha \bullet X \beta$ 的项所对应的项 $A \rightarrow \alpha X \bullet \beta$ 的集合 的闭包

LR(0) Parsing DFA构造: Goto(状态转换函数)

I: a set of items(项集)

X: a symbol (terminal or non-terminal)

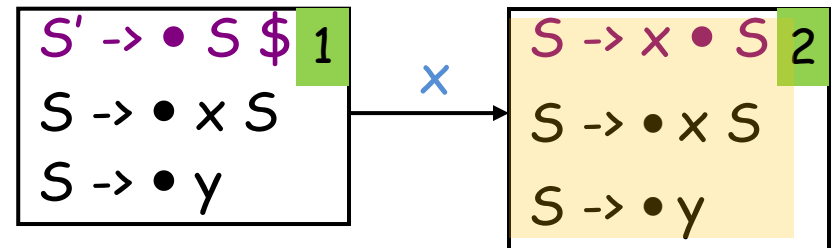
```
Goto(I, X) =  
  set J to the empty set  
  for any item  $A \rightarrow \alpha \cdot X \beta$  in I  
    add  $A \rightarrow \alpha X \cdot \beta$  to J  
  return Closure(J)
```

类似NFA转DFA中计算新的DFA状态

```
Closure(I) =  
  repeat  
    for any item  $A \rightarrow \alpha \cdot X \beta$  in I  
      for any production  $X \rightarrow \gamma$   
         $I \leftarrow I \cup \{X \rightarrow \cdot \gamma\}$   
  until I does not change.  
  return I
```

0: $S' \rightarrow S \$$
1: $S \rightarrow x S$
2: $S \rightarrow y$

项集2 = Goto(项集1, x)



I 是一个项集, X 是一个文法符号, 则Goto(I , X)定义为 I 中所有形如 $A \rightarrow \alpha \cdot X \beta$ 的项所对应的项 $A \rightarrow \alpha X \cdot \beta$ 的集合的闭包

LR(0) Parsing DFA构造: 收敛条件

I: a set of items(项集)

X: a symbol (terminal or non-terminal)

T: the set of states

E: the set of (shift or goto) edges

Initialize T to $\{\text{Closure}(\{S' \rightarrow \bullet S \$\})\}$
Initialize E to empty.

repeat

for each state I in T

for each item $A \rightarrow \alpha \bullet X \beta$ in I

 let J be $\text{Goto}(I, X)$

$T \leftarrow T \cup \{J\}$

$E \leftarrow E \cup \{I \xrightarrow{X} J\}$

until E and T did not change

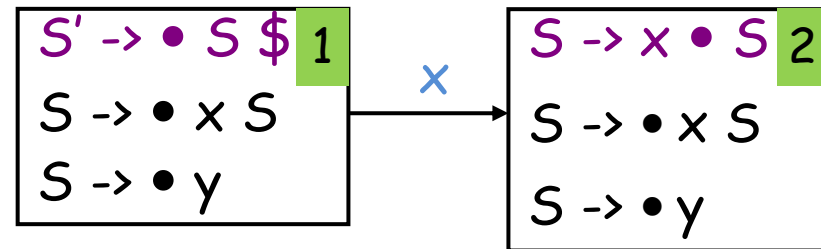
不断增加新状态直到不动点

0: $S' \rightarrow S \$$

1: $S \rightarrow x S$

2: $S \rightarrow y$

• 起始状态 $\text{Closure}(\{S' \rightarrow \bullet S \$\})$



$S' \rightarrow S \bullet \$$ 4

• 不断用Goto函数计算新状态

包含 $S' \rightarrow S \bullet \$$ 的项集所对应的DFA状态为终止状态

LR(0) Parsing DFA构造: 完整算法

I: a set of items

X: a symbol (terminal or non-terminal)

T: the set of states

E: the set of (shift or goto) edges

```
Closure( $I$ ) =  
  repeat  
    for any item  $A \rightarrow \alpha.X\beta$  in  $I$   
      for any production  $X \rightarrow \gamma$   
         $I \leftarrow I \cup \{X \rightarrow .\gamma\}$   
  until  $I$  does not change.  
  return  $I$ 
```

```
Goto( $I, X$ ) =  
  set  $J$  to the empty set  
  for any item  $A \rightarrow \alpha.X\beta$  in  $I$   
    add  $A \rightarrow \alpha X.\beta$  to  $J$   
  return Closure( $J$ )
```

```
Initialize  $T$  to  $\{\mathbf{Closure}(\{S' \rightarrow .S\})\}$   
Initialize  $E$  to empty.
```

```
repeat  
  for each state  $I$  in  $T$   
    for each item  $A \rightarrow \alpha.X\beta$  in  $I$   
      let  $J$  be Goto( $I, X$ )  
       $T \leftarrow T \cup \{J\}$   
       $E \leftarrow E \cup \{I \xrightarrow{X} J\}$   
until  $E$  and  $T$  did not change in this  
iteration
```

1. calculate the new state
2. store the state
3. store the edge

例: LR(0) Parsing DFA构造

• 考虑以下LR(0)文法

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$

Initialize T to $\{\text{Closure}(\{S' \rightarrow \cdot S\})\}$

Initialize E to empty.

repeat

for each state I in T

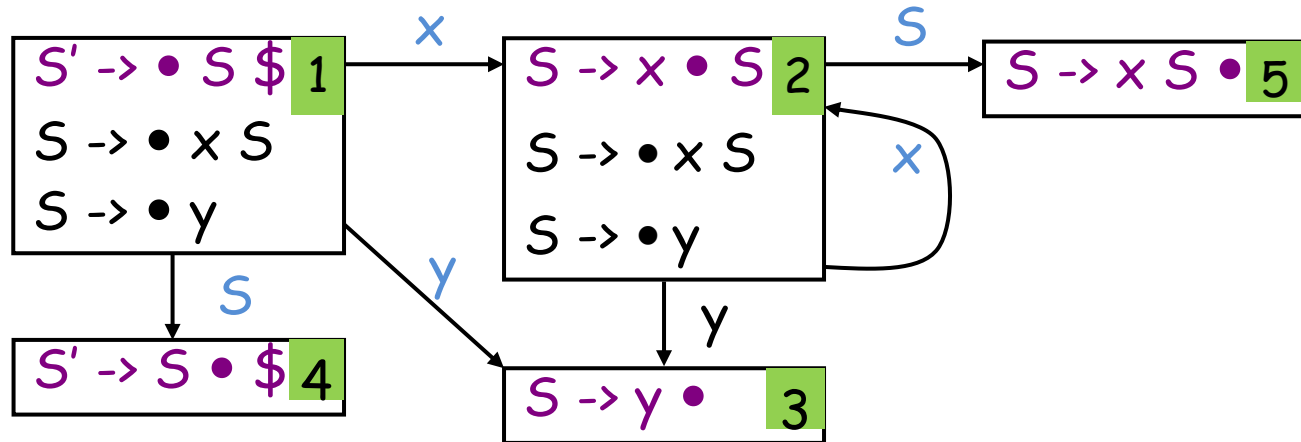
for each item $A \rightarrow \alpha.X\beta$ in I

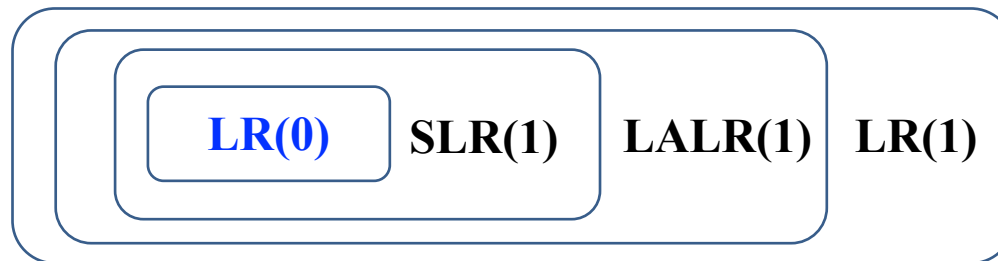
 let J be Goto(I, X)

$T \leftarrow T \cup \{J\}$

$E \leftarrow E \cup \{I \xrightarrow{X} J\}$

until E and T did not change in this iteration





2. LR(0) 分析

- LR(0) Parsing NFA
- LR(0) Parsing DFA
- LR(0) Parsing 分析表
- LR(0) Parsing 过程

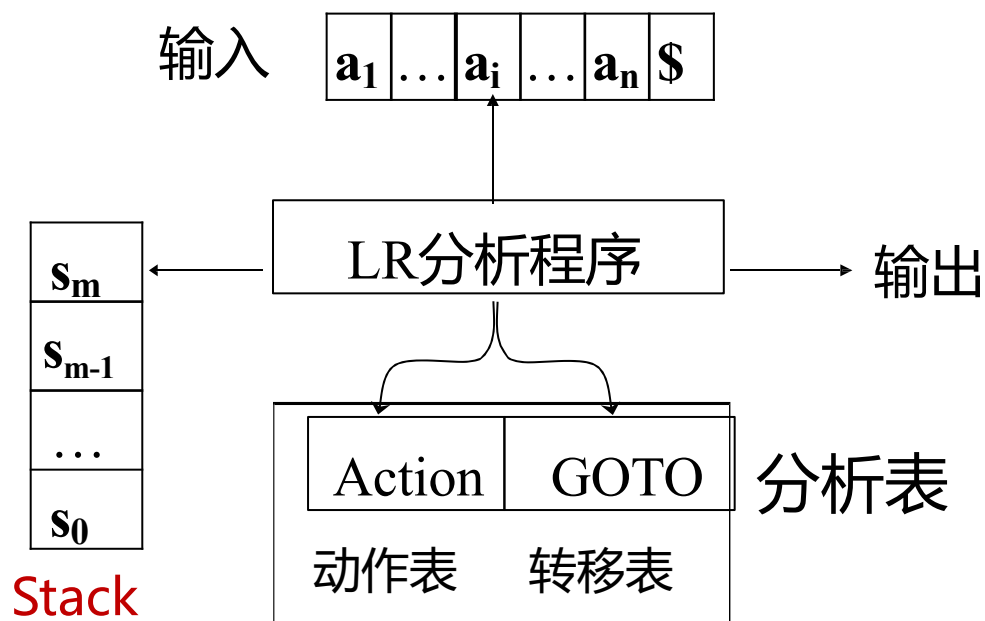
回顾: 基于栈的移进-归约

- **Two components**

- **Symbol Stack:** holds the left-substring α (terminals + nonterminals)

And the “**state stack**”

- **Input stream:** holds the remaining input β (terminals)



- LR系列分析使用相同的表驱动分析程序
- 分析表随文法的不同而各异(如SLR, LR(1))

回顾: 基于栈的移进-归约

- **Four actions**

- **Shift**: push next input (terminal) on to top of **symbol stack**

- **Reduce**:

- Top of symbol stack matches RHS of rule (e.g., $X \rightarrow \mathbf{AB}$)

- 1. pop the RHS from the top of stack (e.g., **pop B A**)

- 2. push the LHS onto the stack (e.g., **push X**)

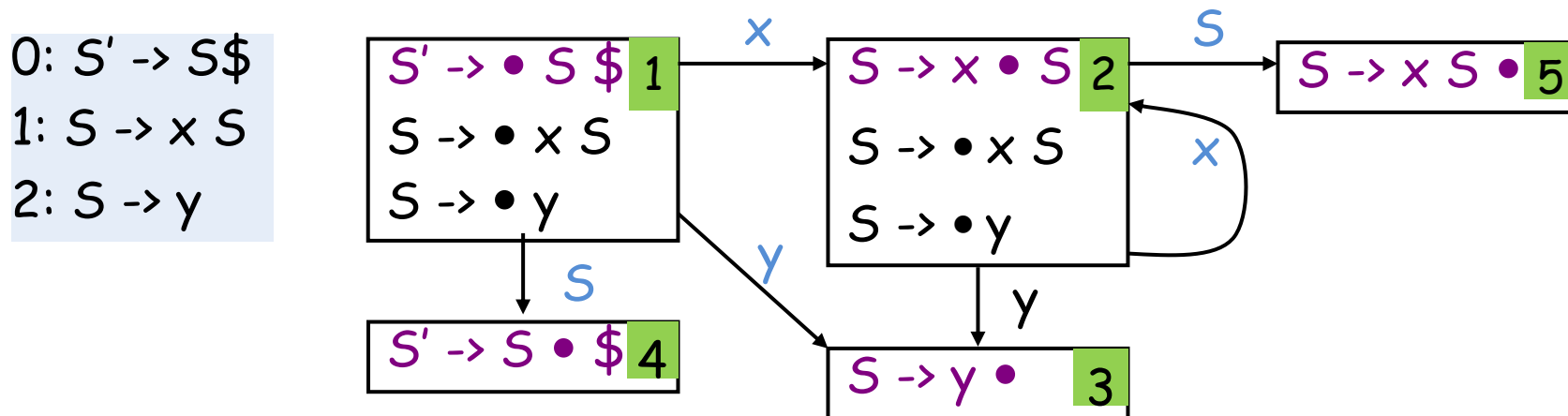
- **Error**

- **Accept**: shift \$ and can reduce what remains on stack to the start symbol!

可以理解为符号栈和状态栈两个都变化，下面我们主要围绕状态栈讲解算（注：龙书中的“stack”既包含符号也包含状态）

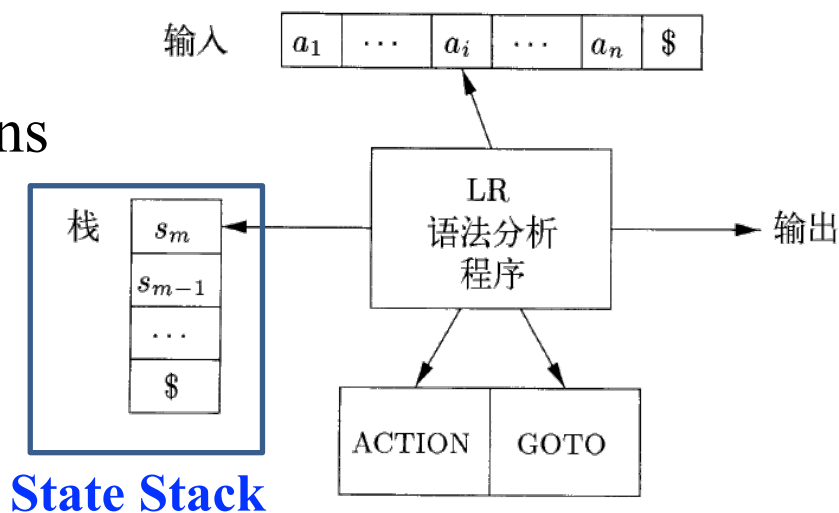
从LR(0) Parsing DFA到语法分析表

- 假设我们已经构造了LR(0) 自动机



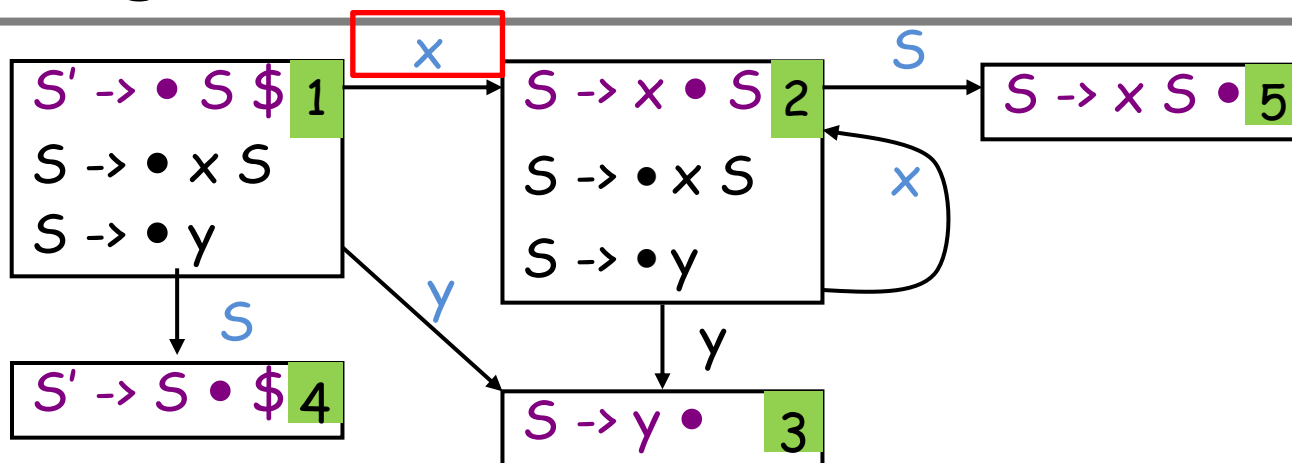
We must create the **Action** and **GOTO** tables to guide four actions

1. Shift
2. Reduce
3. Error
4. Accept



从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$
 1: $S \rightarrow x S$
 2: $S \rightarrow y$



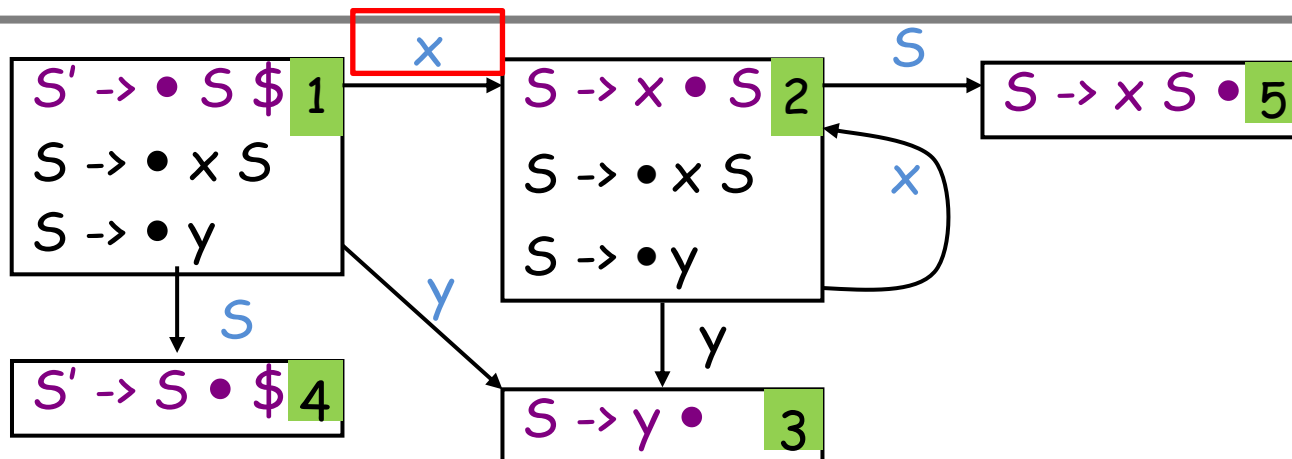
State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

“Summarize” the finite automata

- Action表项的参数：状态*i*，终结符号*a*
 - Tell the parser: what to do in each state (given the lookahead symbol): either shift and go to a new state, or else reduce some production.

从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$
 1: $S \rightarrow x S$
 2: $S \rightarrow y$



State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

“Summarize” the finite automata

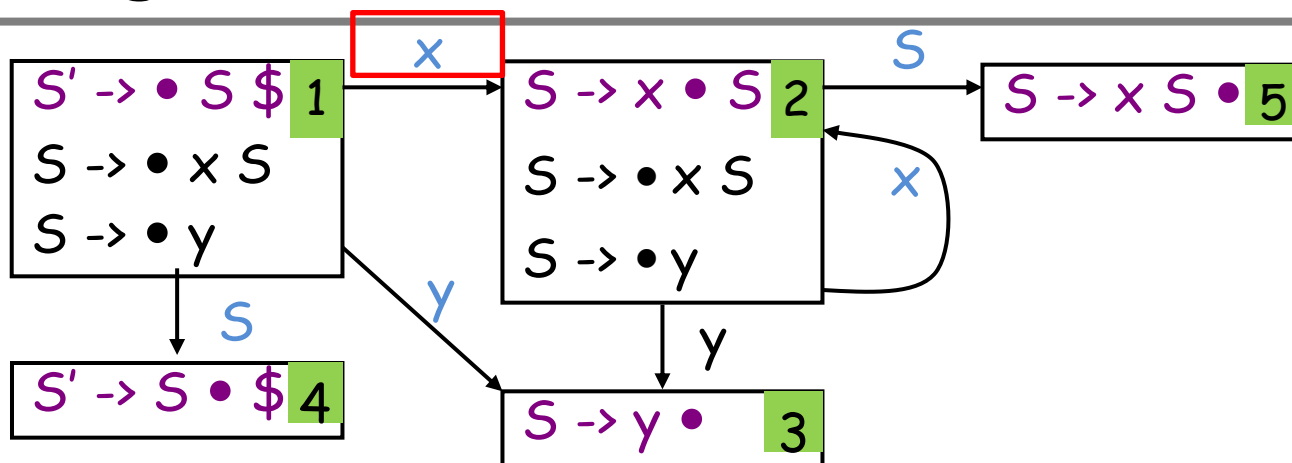
- Action表项的参数: 状态 i , 终结符号 a
 - Tell the parser: what to do in each state (given the lookahead symbol): either shift and go to a new state, or else reduce some production.
- GOTO表项的参数: 状态 i , 非终结符 A
 - Tell the parser: what state to go to when it reduces a production (每次规约所用产生式左边的nonterminal A)

从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$

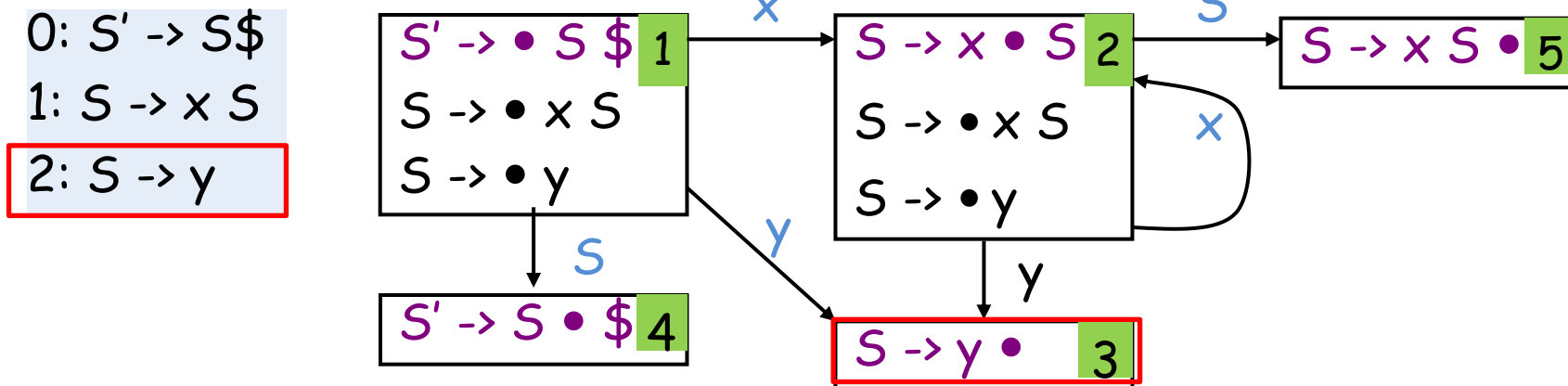


State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Shift: edge labeled with terminal t and from si to sn :

- $T[i, t] = sn$ (shift n)

从LR(0) Parsing DFA到语法分析表



State	Action			GOTO
	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Shift: edge labeled with **terminal t** and from **si** to **sn**:

- $T[i, t] = sn$ (shift n)

Reduce: Item in a state **i** with dot at the end

- $T[i, \text{each terminal}] = rk$ (reduce k)
- **k** is the **index of this production**

$R \leftarrow \{\}$ // 规约动作集合

LR(0)

for each state I in T

for each item $A \rightarrow \alpha.$ in I

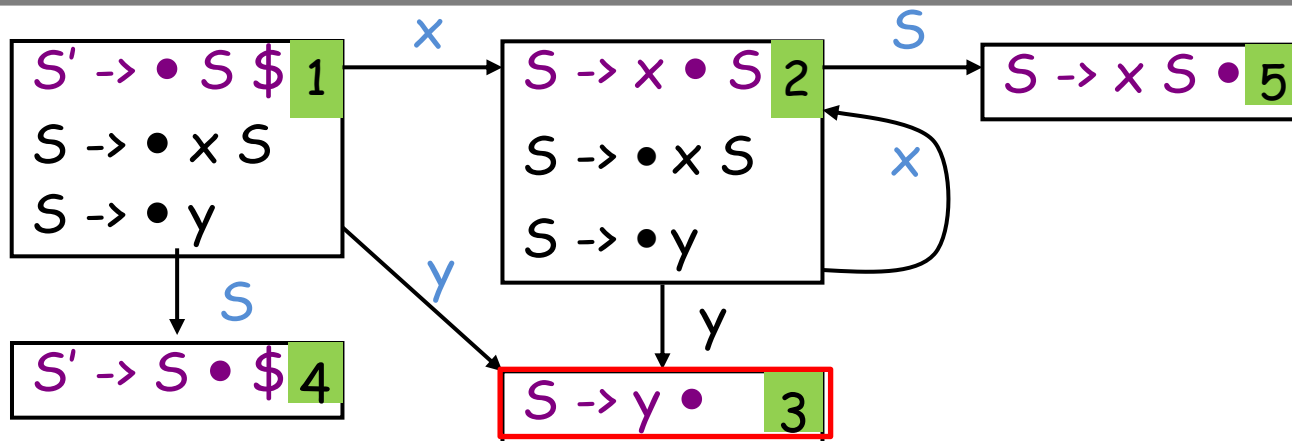
$R \leftarrow R \cup \{(I, A \rightarrow \alpha)\}$

从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



State	Action			GOTO
	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Shift: edge labeled with **terminal t** and from **si** to **sn**:

- $T[i, t] = sn$ (shift n)

Reduce: Item in a state **i** with dot at the end

- $T[i, \text{each terminal}] = rk$ (reduce k)
- **k** is the **index of this production**

当状态i存在以 $\bullet$ 为结尾的项时，选择index为k的产生式进行reduce

如: 状态3存在项 $S \rightarrow y \bullet$

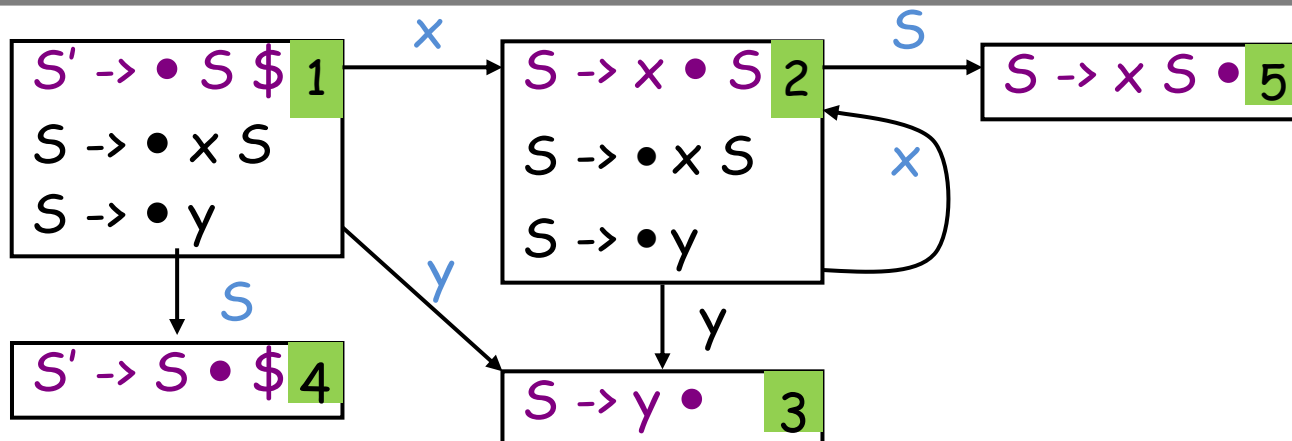
r2表示“根据产生式2: $S \rightarrow y$ ”进行reduce

从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



State	Action			GOTO
	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Shift: edge labeled with **terminal t** and from **si** to **sn**:

- $T[i, t] = sn$ (shift n)

Reduce: Item in a state **i** with dot at the end

- $T[i, \text{each terminal}] = rk$ (reduce k)
- **k** is the **index of this production**

Accept: for each state **i** containing $S' \rightarrow S \cdot \$$:

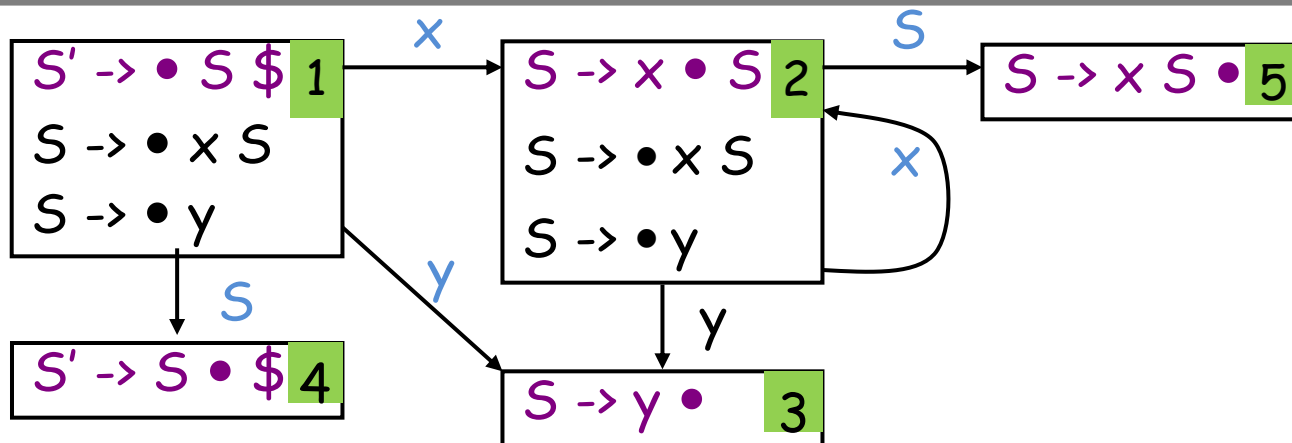
- $T[i, \$] = \text{accept}$

从LR(0) Parsing DFA到语法分析表

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



State	Action			GOTO
	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Shift: edge labeled with **terminal** t and from si to sn :

- $T[i, t] = sn$ (shift n)

Reduce: Item in a state i with dot at the end

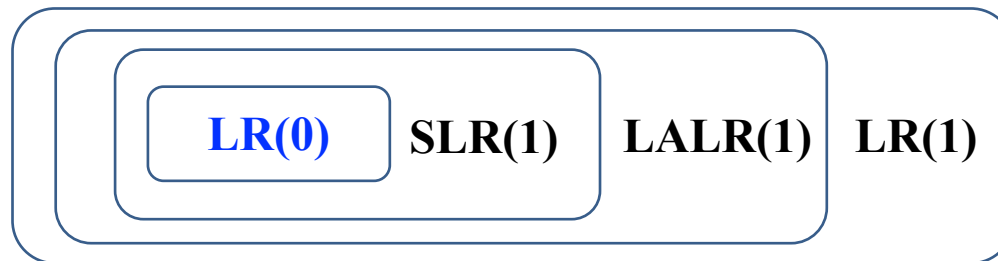
- $T[i, \text{each terminal}] = rk$ (reduce k)
- k is the **index of this production**

Accept: for each state i containing $S' \rightarrow S \cdot \$$:

- $T[i, \$] = \text{accept}$

Goto: edge labeled with **non-terminal** X and from si to sn :

- $T[i, X] = gn$ (goto n)



2. LR(0) 分析

- LR(0) Parsing NFA
- LR(0) Parsing DFA
- LR(0) Parsing 分析表
- LR(0) Parsing 过程

回顾: 基于栈的移进-归约

- **Four actions (关于符号栈的变化)**
 - **Shift**: push next input (terminal) on to top of stack
 - **Reduce**:
 - Top of stack should match RHS of rule (e.g., $X \rightarrow A B$)
 - pop the RHS from the top of stack (e.g., **pop** $B A$)
 - push the LHS onto the stack (e.g., **push** X)
 - **Error**
 - **Accept**: shift \$ and can reduce what remains on stack to the start symbol!

可以理解为符号栈和状态栈两个都变化，下面我们主要围绕状态栈讲解算（注：龙书中的“stack”既包含符号也包含状态）

LR 语法分析算法(主要围绕状态栈的变化)

State Stack: LR Parsing DFA状态 ; Input Stream: 输入

Look up **top stack state** and **input symbol**, to get action

Shift(n)	1. Advance input one token; 2. Push n on stack.
Reduce(k)	1. Pop stack as many times as the number of symbols on the RHS of rule k; 2. Let X be the LHS symbol of rule k. In the state now on top of stack, look up X to get “goto n”; Push n on top of stack.
Accept	Stop parsing, report success.
Error	Stop parsing, report failure.

Shift(n) : 移进1个token;
把新状态 n 压入栈

- This is a **general algorithm** and can be used by other LR parsing
- For **LR(0)**, we do not need to look up input symbol to know whether we should shift or reduce

LR 语法分析算法

State Stack: LR Parsing DFA状态 ; Input Stream: 输入

Look up **top stack state** and **input symbol**, to get action

Shift(n)	1. Advance input one token; 2. Push n on stack.
Reduce(k)	1. Pop stack as many times as the number of symbols on the RHS of rule k; 2. Let X be the LHS symbol of rule k. In the state now on top of stack, look up X to get “goto n”; Push n on top of stack.
Accept	Stop parsing, report success.
Error	Stop parsing, report failure.

Reduce(k) // k-th rule $X \rightarrow \alpha$
pop $|\alpha|$ states
push GOTO[top(stack), X]

- This is a **general algorithm** and can be used by other LR parsing
- For **LR(0)**, we do not need to look up input symbol to know whether we should shift or reduce

LR 语法分析算法

- 输入：文法的LR语法分析表，输入串 w

令 s 是栈顶状态, a 是 w \$的符号;

while (1) { // 一开始 s 为Parsing DFA的状态1

if (**Action**[s, a] = “shift s' ”) {

 将 s' 压入栈内 ;

 让 a 为下一个输入符号 ; // 移进

} else if (**Action**[s, a] = “reduce $A \rightarrow \beta$ ”) {

 从栈顶弹出 $|\beta|$ 个状态 ;

 令 s' 是当前栈顶状态, 把**GOTO**[s', A]入栈 ;

} else if (**Action**[s, a] = “Accept”) **break**;

else error(); // 后面会讲;

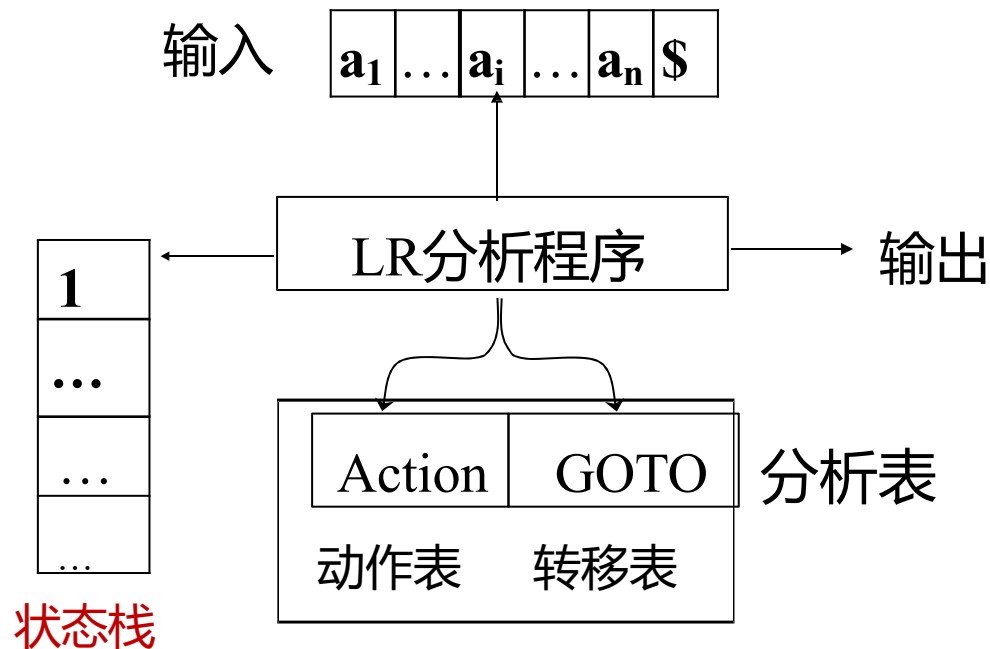
}

例: LR(0) 分析的Shift动作

0: $S' \rightarrow S\$$
1: $S \rightarrow x S$
2: $S \rightarrow y$

State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

注: 算法主要考虑**状态栈**,
符号栈信息可相应状态中获取



Shift: 假设读入y. 根据 $\text{Action}(1, y) = s3$

- push状态3到状态栈的栈顶

例: LR(0) 分析的Reduce动作

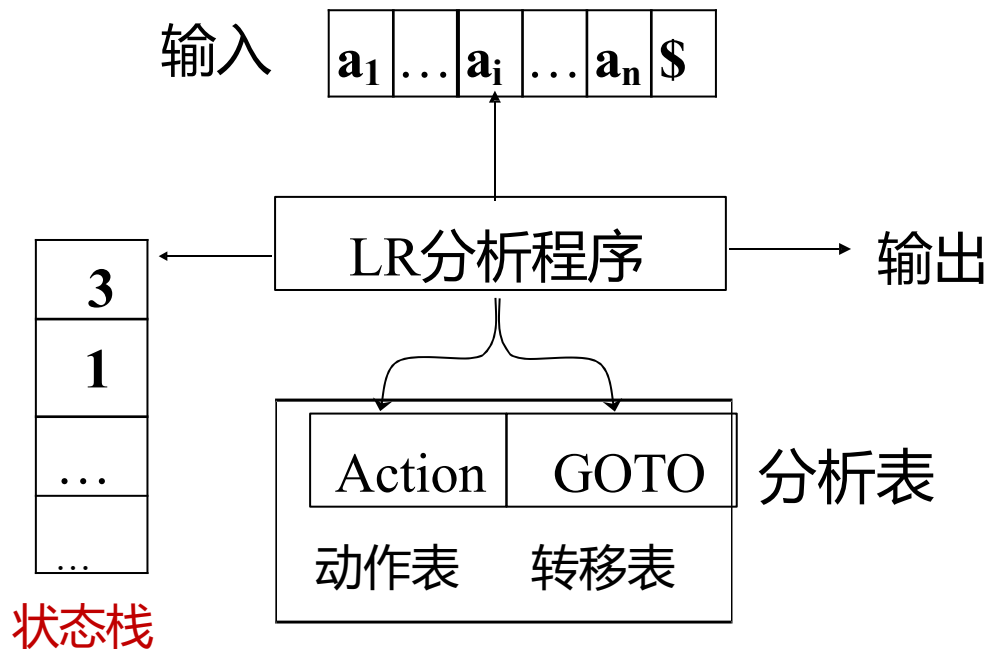
Reduce(k) // k-th rule $X \rightarrow \alpha$
pop $|\alpha|$ states
push GOTO[top(stack), X]

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$

State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	



Reduce: 假设下一Token是x, 根据 $T(3, x) = r2$

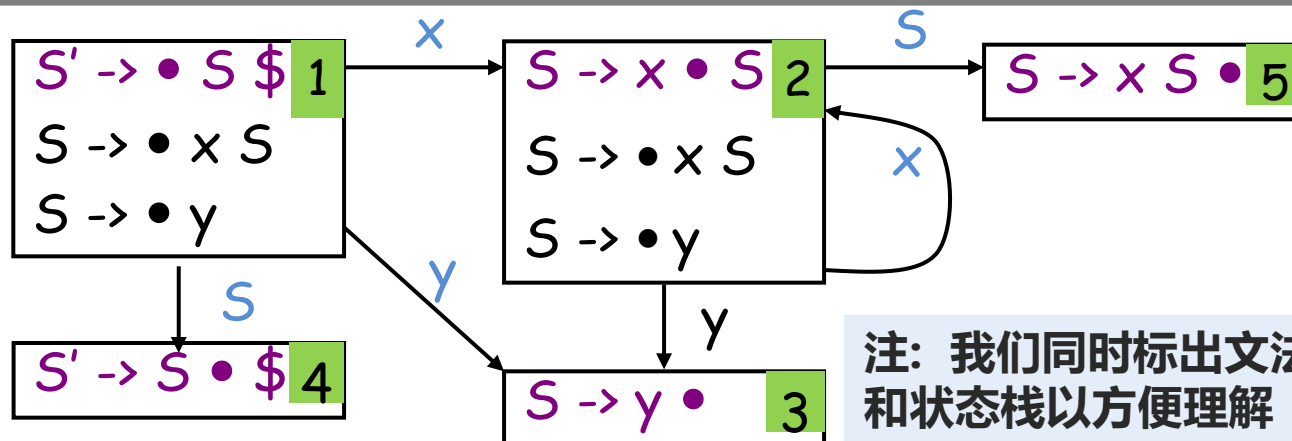
1. Pop状态栈顶 $|RHS(S \rightarrow y)| = 1$ 个元素
2. 状态栈顶变成1, 根据 $GOTO(1, S) = g4$, push状态4到状态栈上

例: 完整的LR(0)语法分析

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



注: 我们同时标出文法符号栈和状态栈以方便理解

	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2

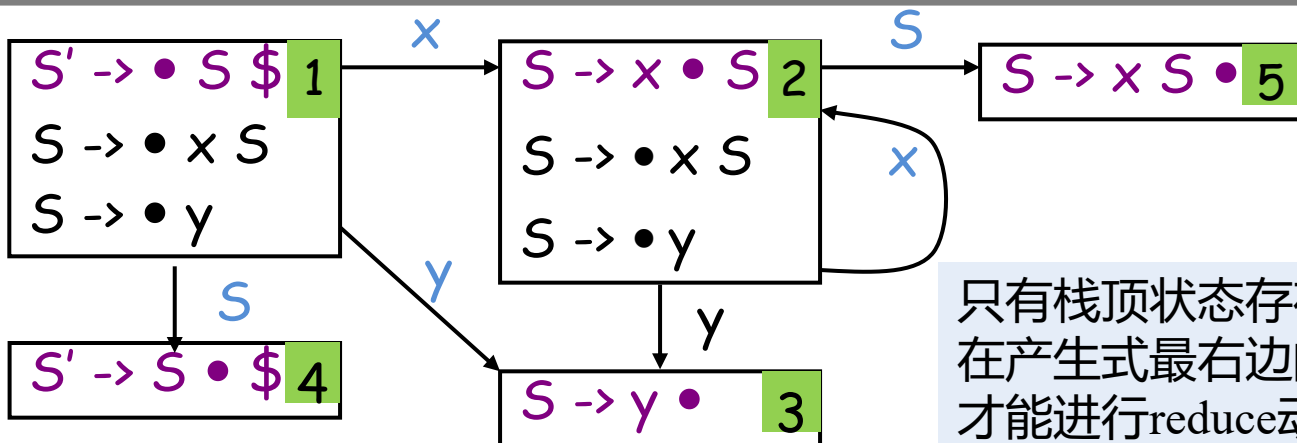
Shift 2: 读入x (移进), 状态栈压入2

例: 完整的LR(0)语法分析

0: $S' \rightarrow S \$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2

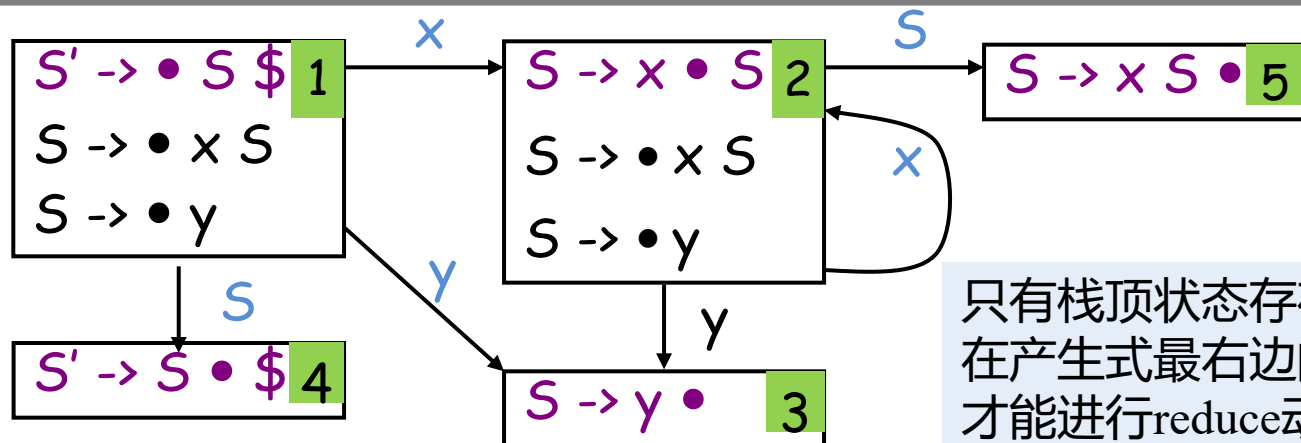
Shift 2: 读入x, 状态栈压入2

例: 完整的LR(0)语法分析

0: $S' \rightarrow S \$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2
1 2 2	x x	y \$	shift 3

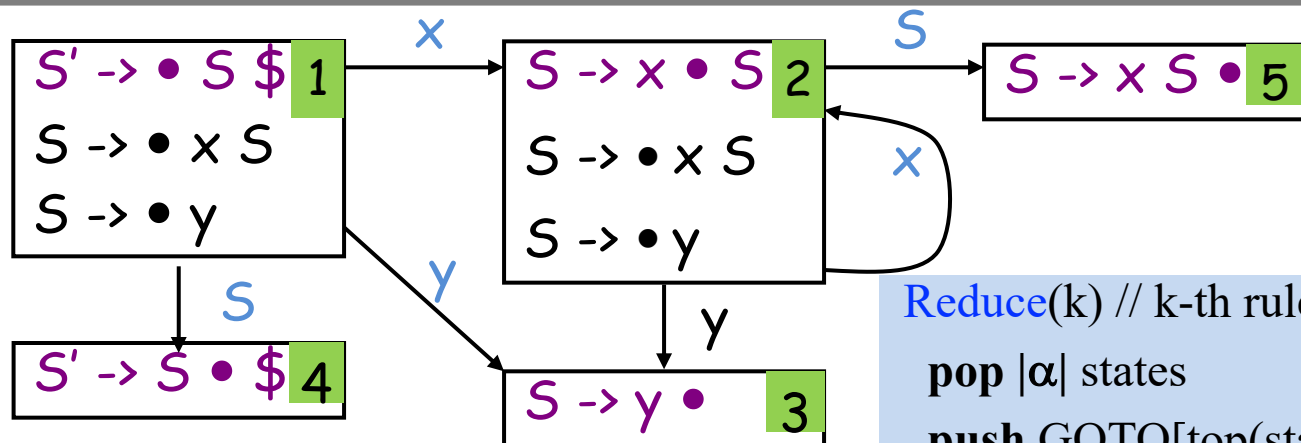
Shift 3: 读入y, 状态栈压入3

例: 完整的LR(0)语法分析

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



Reduce(k) // k-th rule $X \rightarrow \alpha$
 pop $|\alpha|$ states
 push GOTO[top(stack), X]

	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2
1 2 2	x x	y \$	shift 3
1 2 2 3	x x y	\$	reduce 2 ($S \rightarrow y$)

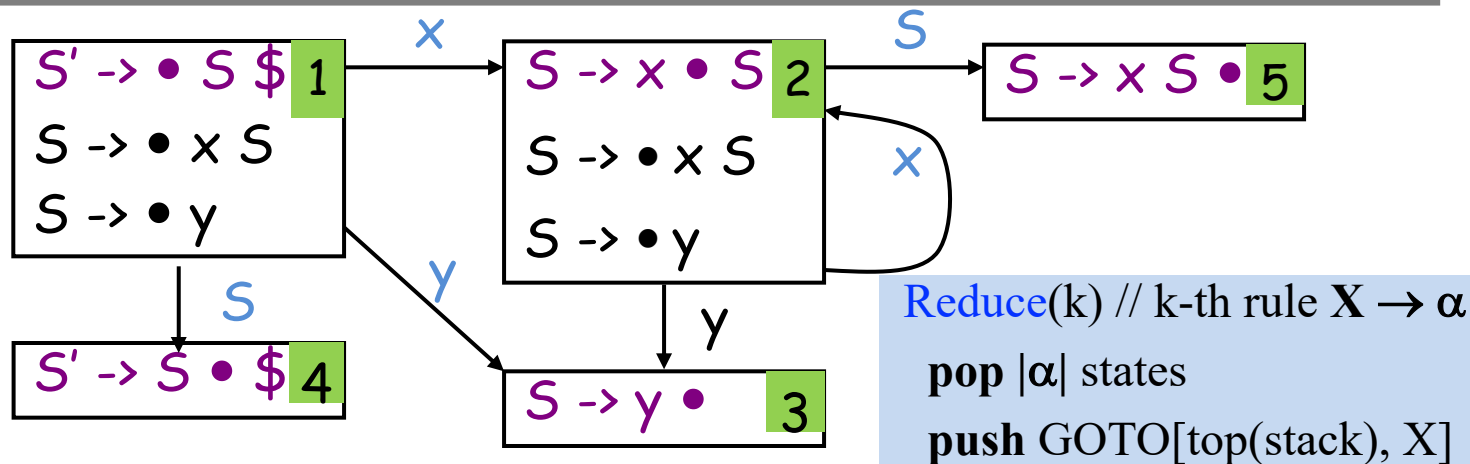
Reduce动作: 1. 状态栈弹出状态3($|RHS|$ 个状态); 2. 根据Goto(2, S) = 5 压入状态5 ⁷⁷

例: 完整的LR(0)语法分析

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2
1 2 2	x x	y \$	shift 3
1 2 2 3	x x y	\$	reduce 2 ($S \rightarrow y$)
1 2 2 5	x x S	\$	reduce 1 ($S \rightarrow x S$)

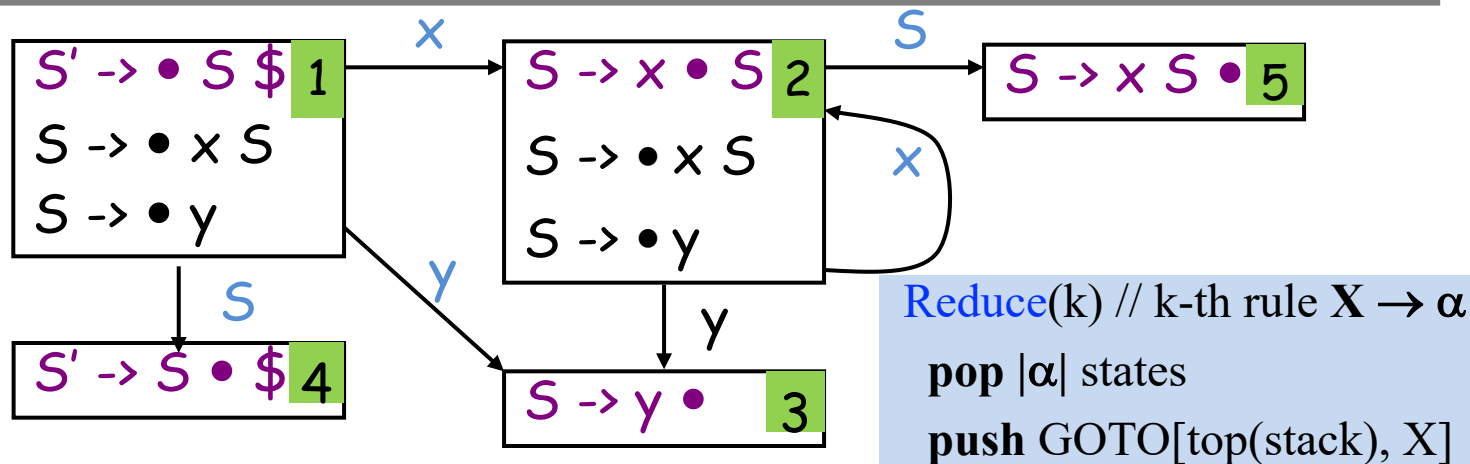
Reduce动作: 1. 状态栈弹出5和2($|\text{RHS}|$ 个状态); 2. 根据 $\text{Goto}(2, S) = 5$ 压入状态5

例: 完整的LR(0)语法分析

0: $S' \rightarrow S\$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2
1 2 2	x x	y \$	shift 3
1 2 2 3	x x y	\$	reduce 2 ($S \rightarrow y$)
1 2 2 5	x x S	\$	reduce 1 ($S \rightarrow x S$)
1 2 5	x S	\$	reduce 1 ($S \rightarrow x S$)

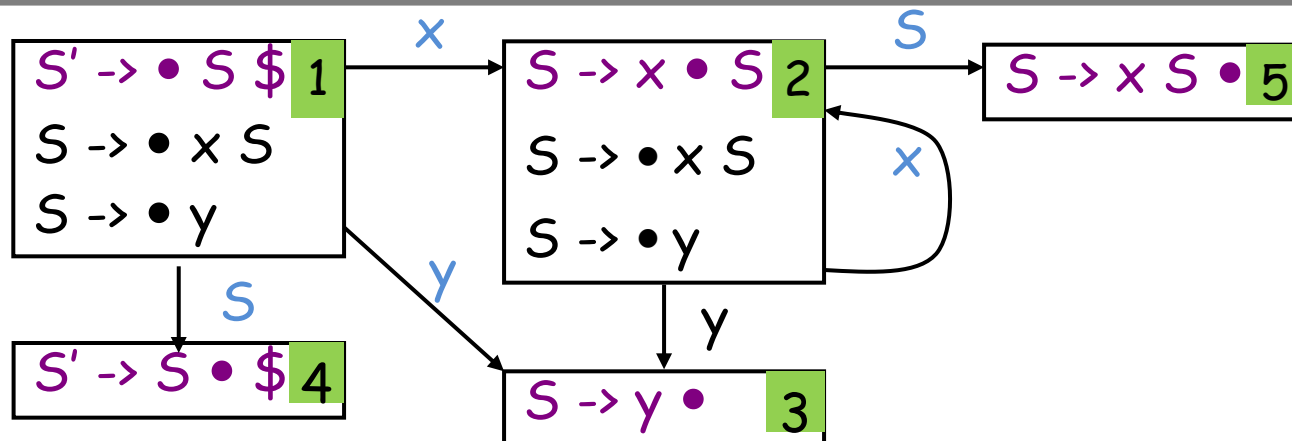
Reduce: 1. 状态栈弹出5和2($|\text{RHS}|$ 个状态); 2. 根据 $\text{Goto}(1, S) = 4$ 压入状态4

例: 完整的LR(0)语法分析

0: $S' \rightarrow S \$$

1: $S \rightarrow x S$

2: $S \rightarrow y$



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

Stack (states)	Stack (symbols)	Input	Action
1		x x y \$	shift 2
1 2	x	x y \$	shift 2
1 2 2	x x	y \$	shift 3
1 2 2 3	x x y	\$	reduce 2 ($S \rightarrow y$)
1 2 2 5	x x S	\$	reduce 1 ($S \rightarrow x S$)
1 2 5	x S	\$	reduce 1 ($S \rightarrow x S$)
1 4	S	\$	accept

Accept: 栈顶状态4报包含项 $S' \rightarrow S \bullet \$$, 因此Accept

课后练习: LR(0)分析: DFA构造

0: $S' \rightarrow S\$$
 1: $S \rightarrow (L)$
 2: $S \rightarrow x$
 3: $L \rightarrow S$
 4: $L \rightarrow L, S$

Initialize T to $\{\text{Closure}(\{S' \rightarrow .S\$ \})\}$

Initialize E to empty.

repeat

for each state I in T

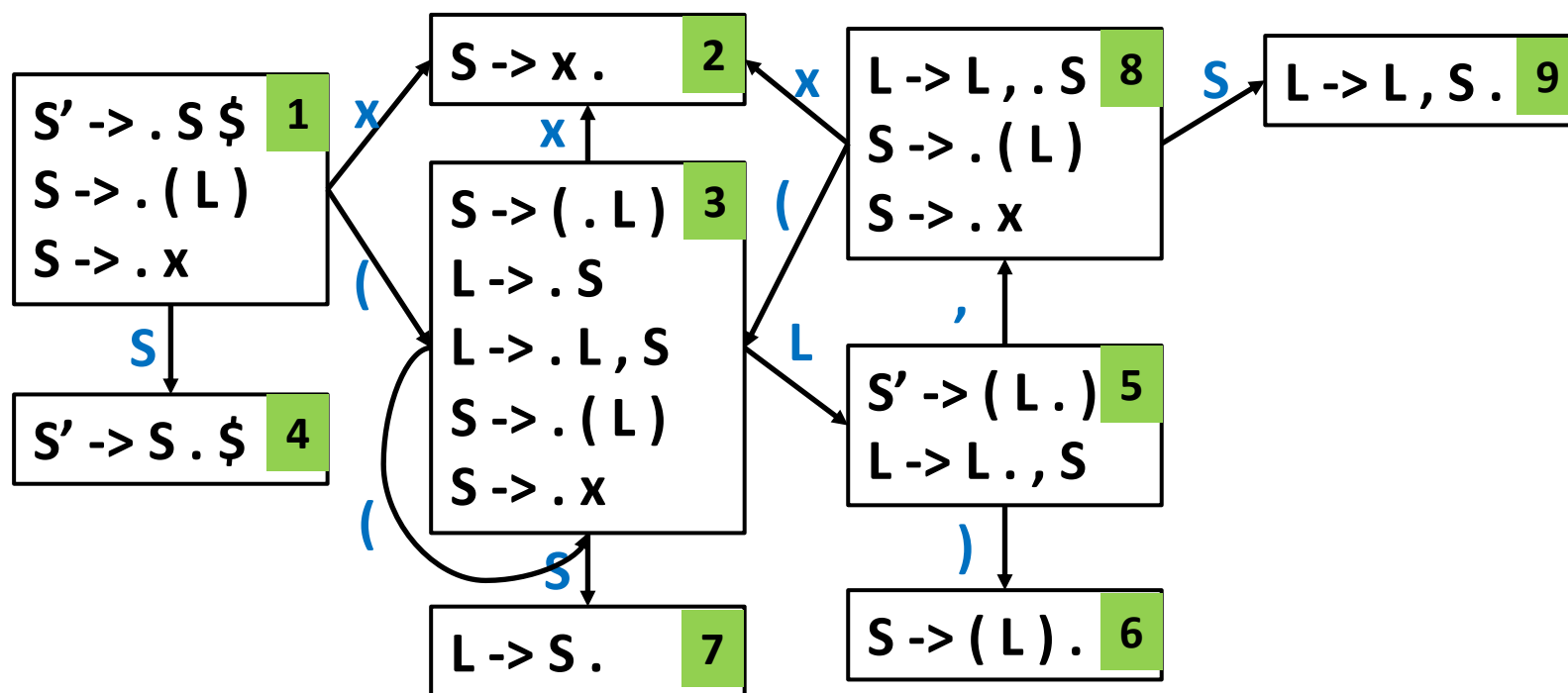
for each item $A \rightarrow \alpha.X\beta$ in I

 let J be Goto(I, X)

$T \leftarrow T \cup \{J\}$

$E \leftarrow E \cup \{I \xrightarrow{X} J\}$

until E and T did not change in this iteration



课后练习: LR(0)分析-Parsing DFA到语法分析表

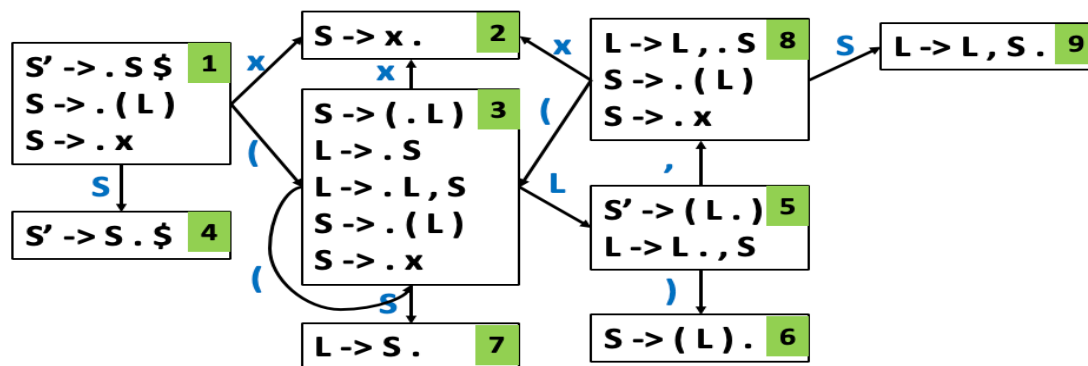
0: $S' \rightarrow S\$$

1: $S \rightarrow (L)$

2: $S \rightarrow x$

3: $L \rightarrow S$

4: $L \rightarrow L, S$



	Action					GOTO	
	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

课后练习: LR(0)分析-语法分析表的使用

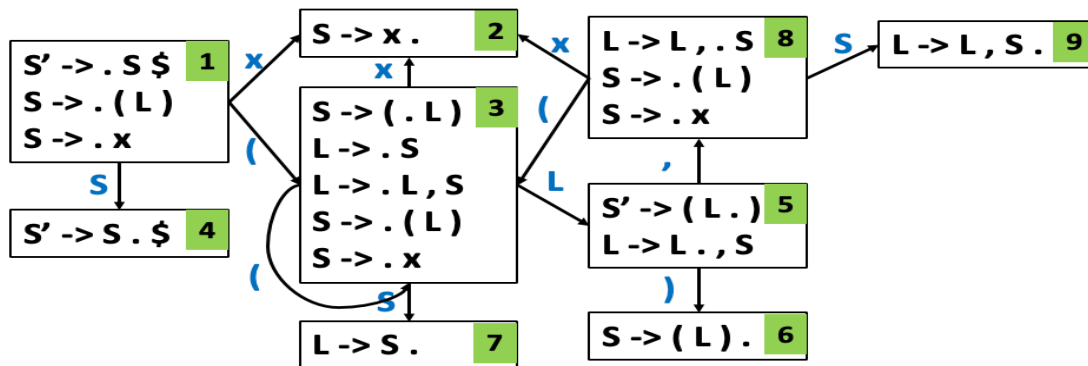
0: $S' \rightarrow S\$$

1: $S \rightarrow (L)$

2: S $\rightarrow$ x

3: L $\rightarrow$ S

4: L $\rightarrow$ L, S



	Action					GOTO	
	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

[illegible]

课后练习: LR(0)分析-语法分析表的使用

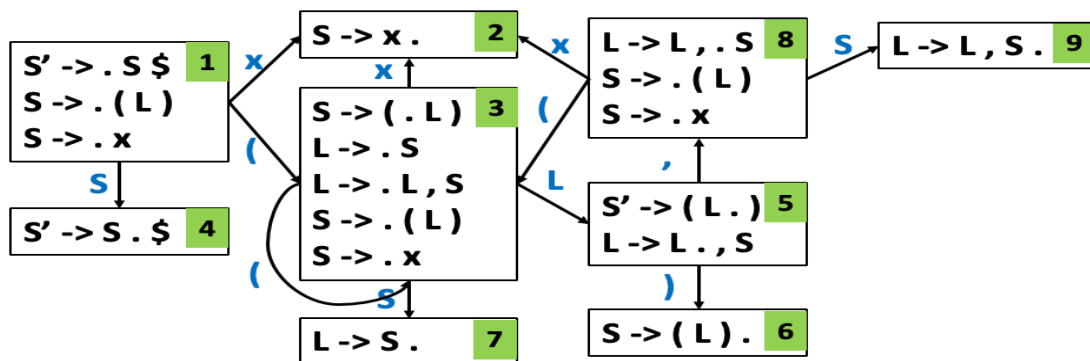
0: $S' \rightarrow S\$$

1: $S \rightarrow (L)$

2: $S \rightarrow x$

3: $L \rightarrow S$

4: $L \rightarrow L, S$



	Action					GOTO	
	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

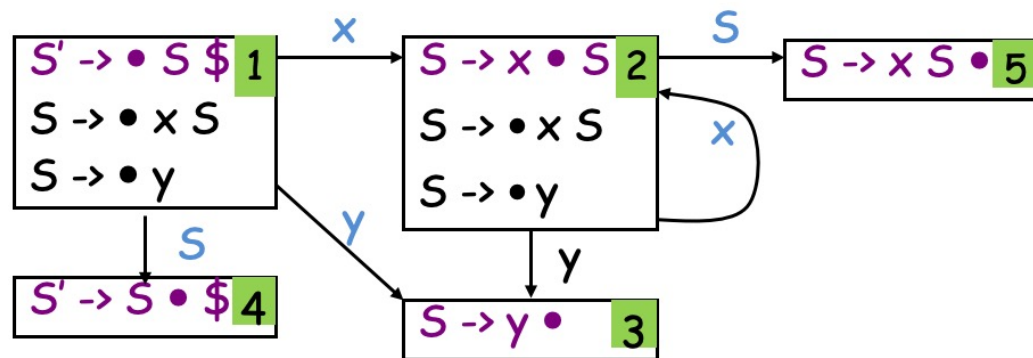
Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	reduce 1 $S \rightarrow (L)$
1	S	\$	goto 4
1, 4	S	\$	accept

思考题: 关于LR(0)中的0

• 为什么说LR(0)没有Lookahead?

- LR(0) Item中没有lookahead terminal等信息
- “是否归约、选择哪个产生式规约”仅由栈顶状态决定

State	Action			GOTO
	x	y	\$	
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

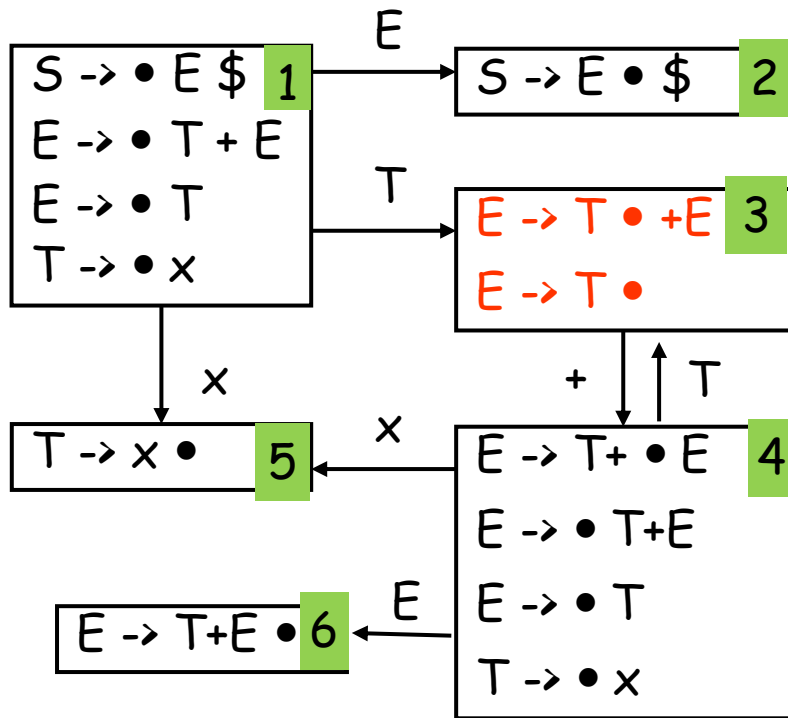


- LR(0)中所有的 reduce 操作实际上是与状态绑定的，而不是与 (状态, 输入) 对绑定的。因此我们在读取当前符号之前，就知道应该 reduce 还是shift 了
- 而如果是 shift，则可认为是读入当前符号以后再做判断，也就是说从理论上讲我们没有(或者说不必)做任何 look-ahead

LR(0)分析的问题/局限性

0: $S \rightarrow E\$$
1: $E \rightarrow T+E$
2: $E \rightarrow T$
3: $T \rightarrow x$

- For every item of the form: $X \rightarrow \alpha \bullet$, **blindly** reduce α to X
- May lead to conflicts $\rightarrow$ failed?



A shift-reduce conflict (on state 3)!

s\t	Action			GOTO	
	x	+	\$	E	T
1	s5			g2	g3
2			accept		
3	r2	s4, r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

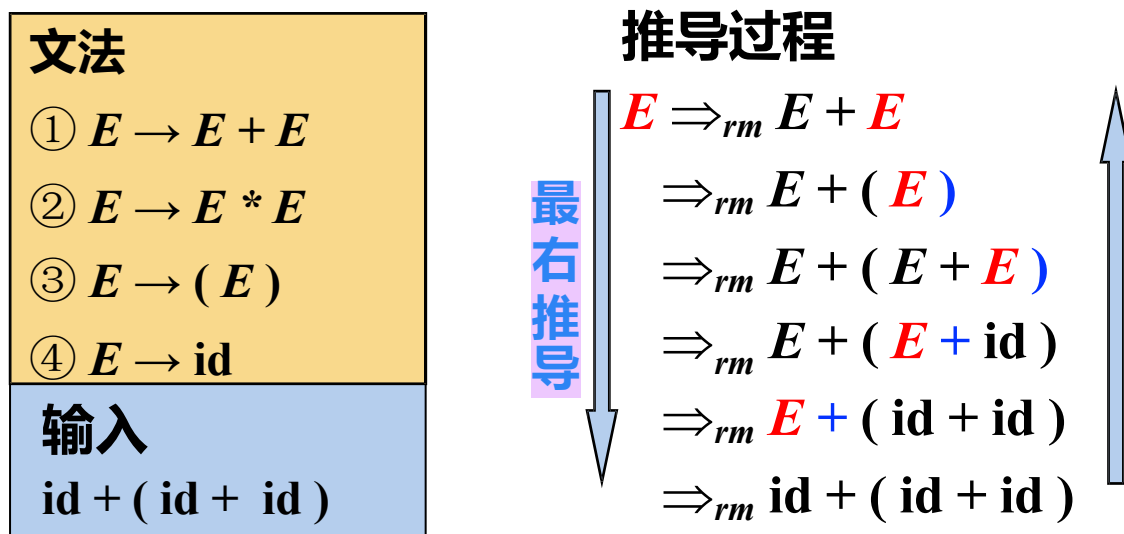
由于LR(0)分析器不会查看下一个输入符号，所以没有足够的上下文来做出更正确(或者说更“聪明”)的决定。



3. SLR(1)分析

SLR(1) Parsing思路

- 直观想法: 利用更多信息来指导规约操作
- 回顾: “LR分析应该是最右推导的逆过程”



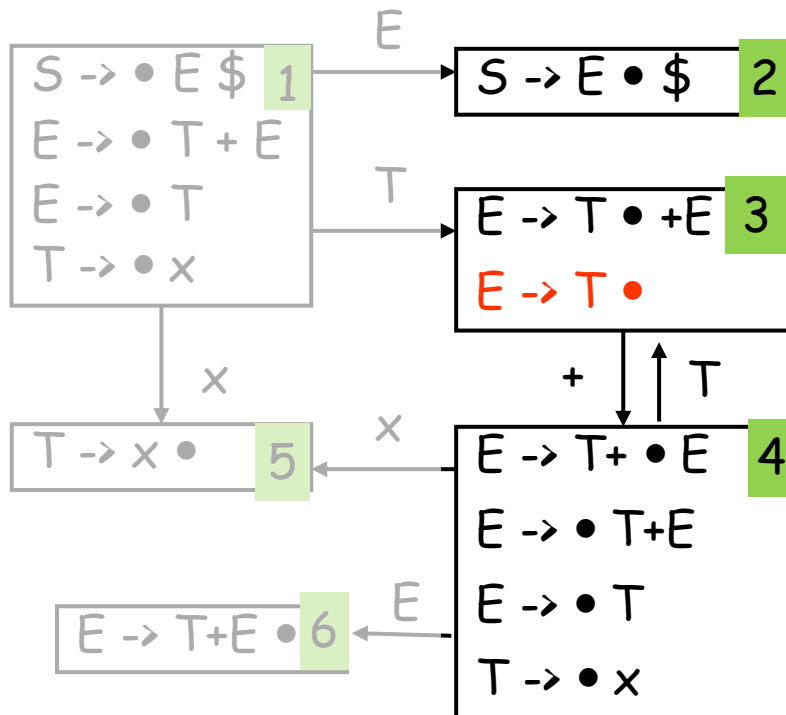
- 因此, 每步归约都应该满足 $t \in \text{Follow}(E)$
 - E 是用来归约的产生式的左部
 - t 是 Next Token (Lookahead)

$$\text{Follow}(A) = \{a \mid S \Rightarrow^* \dots Aa\dots, a \in T\}$$

例: SLR(1) Parsing思路

0: $S \rightarrow E\$$
 1: $E \rightarrow T+E$
 2: $E \rightarrow T$
 3: $T \rightarrow x$

- What are the valid next tokens for r2?
 - Idea:** we can choose the reduce action only if the **next input token $t \in \text{Follow}(E) = \{\$ \}$**
 - Only $T[3, \$]$ can be r2!**



	Action			GOTO
State	x	y	\$	S
1	s2	s3		g4
2	s2	s3		g5
3	r2	r2	r2	
4			accept	
5	r1	r1	r1	

SLR(1) Parsing DFA和分析表

- SLR(1) Parsing DFA和LR(0)的相同
- 主要区别: 构造分析表时的归约动作
 - SLR: Put reduce actions into the table only where indicated by the FOLLOW set.

```
 $R \leftarrow \{\}$  // 规约动作集合 SLR(1)  
for each state  $I$  in  $T$   
  for each item  $A \rightarrow \alpha.$  in  $I$   
    for each token  $X$  in Follow( $A$ )  
       $R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$ 
```

```
 $R \leftarrow \{\}$  // 规约动作集合 LR(0)  
for each state  $I$  in  $T$   
  for each item  $A \rightarrow \alpha.$  in  $I$   
     $R \leftarrow R \cup \{(I, A \rightarrow \alpha)\}$ 
```

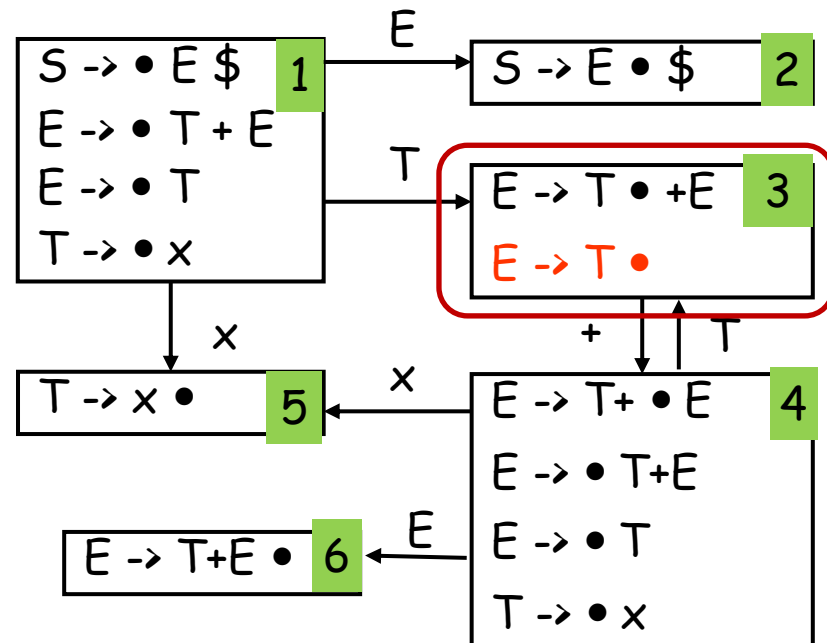
I : the row,
 X : the column
 $A \rightarrow \alpha$: the production

例: 构建SLR(1)分析表

0: $S \rightarrow E\$$
 1: $E \rightarrow T+E$
 2: $E \rightarrow T$
 3: $T \rightarrow x$

Follow (E) = { \$ }

Follow (T) = { +, \$ }



考虑状态3的 $E \rightarrow T$. (用“—”划掉的是按LR(0)填的)

s\t	Action			GOTO	
	x	+	\$	E	T
1	s5			g2	g3
2			accept		
3	r2	s4, r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

$R \leftarrow \{ \}$ // 规约动作集合

SLR(1)

for each state I in T

for each item $A \rightarrow \alpha \cdot$ in I

for each token X in **Follow(A)**

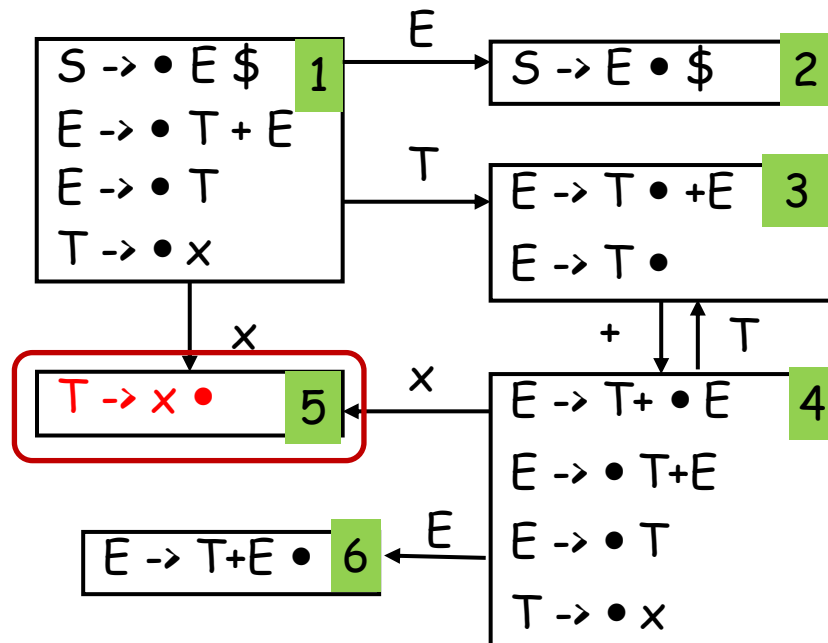
$R \leftarrow R \cup \{ (I, X, A \rightarrow \alpha) \}$

例: 构建SLR(1)分析表

0: $S \rightarrow E\$$
 1: $E \rightarrow T+E$
 2: $E \rightarrow T$
 3: $T \rightarrow x$

Follow (E) = {\$}

Follow (T) = {+, \$}



考虑状态5的 $T \rightarrow x$.

s\t	Action			GOTO	
	x	+	\$	E	T
1	s5			g2	g3
2			accept		
3	r2	s4, r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

$R \leftarrow \{\}$ // 规约动作集合

SLR(1)

for each state I in T

for each item $A \rightarrow \alpha.$ in I

for each token X in Follow(A)

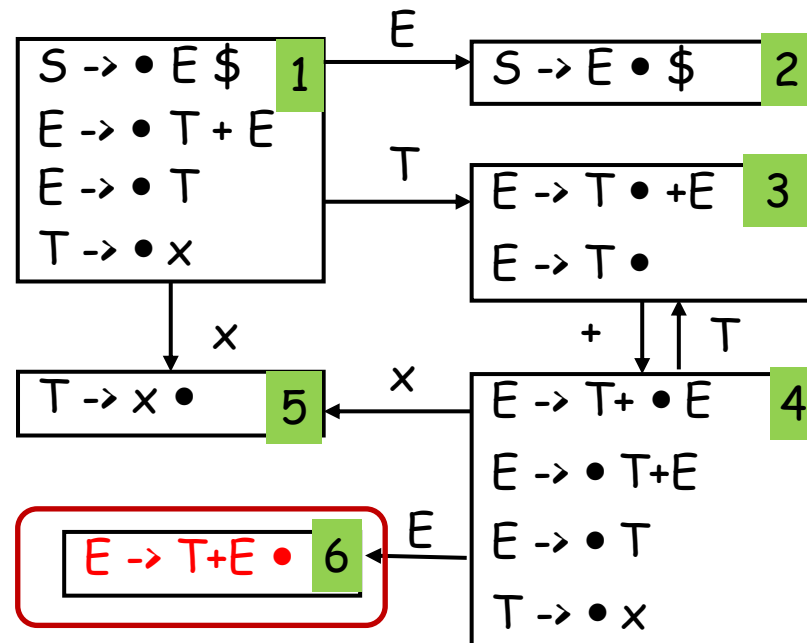
$R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$

例: 构建SLR(1)分析表

0: $S \rightarrow E\$$
 1: $E \rightarrow T+E$
 2: $E \rightarrow T$
 3: $T \rightarrow x$

Follow (E) = { $\$$ }

Follow (T) = {+, $\$$ }



考虑状态6的 $E \rightarrow T + E$.

s\t	Action			GOTO	
	x	+	\$	E	T
1	s5			g2	g3
2			accept		
3	r2	s4, r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

$R \leftarrow \{\}$ // 规约动作集合

SLR(1)

for each state I in T

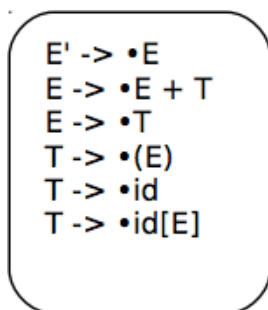
for each item $A \rightarrow \alpha \cdot$ in I

for each token X in Follow(A)

$R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$

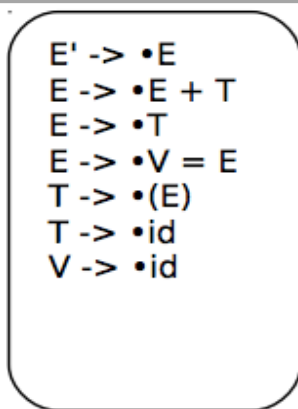
SLR(1)的问题/局限

- In the SLR(1) parser, it is allowable for there to be both shift and reduce items in the same state as well as multiple reduce items
 - The SLR(1) parser will be able to determine which action to take as long as the Follow sets are **disjoint** [可区分即可]



移入还是归约?

- 下一token在T的Follow集就归约;
- 否则, 就不归约而移入



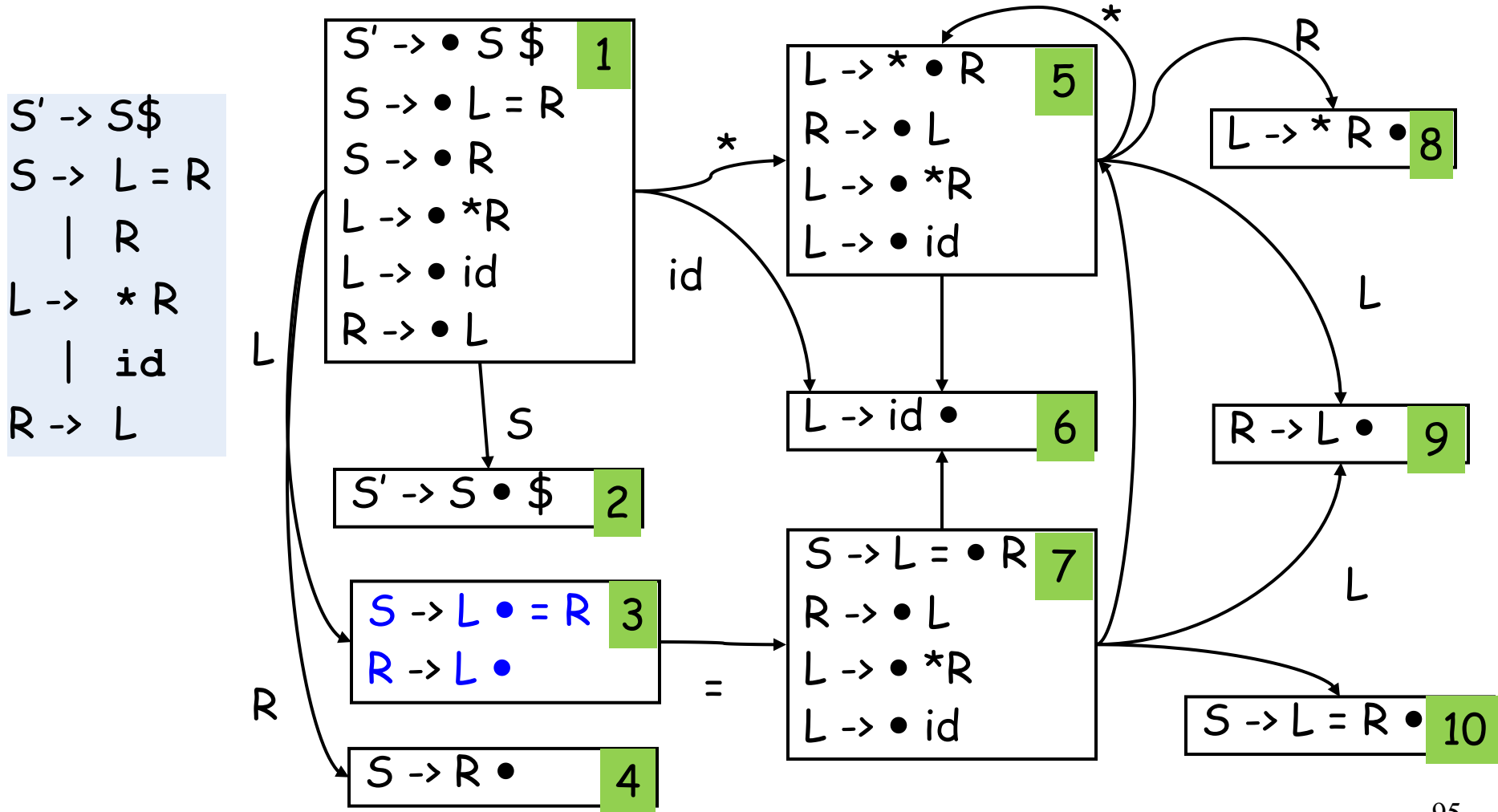
归约到T还是V?

- 下一token在T的Follow集就归约到T;
- 在V的Follow集就归约到V



例: SLR(1)的问题/局限

- Follow(R) = Follow(L)
- Thus, there exists shift-reduce conflict in state 3



SLR(1)问题/局限的解决

- SLR技术解决冲突的方法

- 按照 $A \rightarrow \alpha$ 进行归约的条件是：下一个输入符号 x 可以在某个句型中跟在 A 之后，也就是 $x \in \text{Follow}(A)$

- “LR分析过程应该是最右推导的逆”

- 每一步都应该是最右句型

- 如果 βAx 不是任何最右句型的前缀，那么即使 x 在某个句型中跟在 A 之后，仍不应该按 $A \rightarrow \alpha$ 归约
(利用Follow集可看作“合理但可能不够精确的近似”)

进行归约的条件更加严格才能进一步降低冲突的可能



Thank you all for your attention

Top-down vs. Bottom-up

Matched	Stack (unmatched)	Input	Action
	E \$	int * int \$	$E \rightarrow TE'$
	T E' \$	int * int \$	$T \rightarrow \text{int } T'$
int	int T' E' \$	int * int \$	match
int	T' E' \$	* int \$	$T' \rightarrow *T$
int	* T E' \$	* int \$	match
int *	T E' \$	int \$	$T \rightarrow \text{int } T'$
int *	int T' E' \$	int \$	match
int * int	T' E' \$	\$	$T' \rightarrow \epsilon$
int * int	E' \$	\$	$E' \rightarrow \epsilon$
int * int	\$	\$	Halt-accept

Top-down

Step	Operation
int * int + int	Shift
int * int + int	Shift
int * int + int	Shift
int * int + int	Reduce $T \rightarrow \text{int}$
int * T + int	Reduce $T \rightarrow \text{int} * T$
T + int	Shift
T + int	Shift
T + int	Reduce $T \rightarrow \text{int}$
T + T	Reduce $E \rightarrow T$
T + E	Reduce $E \rightarrow T + E$
E #	

Bottom-up